UNIVERSITY OF PLYMOUTH

In collaboration with Peninsula College

School of Technology & Engineering

BSc (Hons) Computer Science (Cyber Security)

(N/0613/6/0002)(04/2027)(MQA/PA15604)

MAL2020 – Computing Group Project (FINAL)

Author:

BSCS2509254

BSCS2509259

BSCS2509260

BSCS2509261

Module Leader:

Mr. Kong Kok Wah @ Eric

Mar 25, 2026

Table of Contents

Chapter 1 : Introduction 6
1.1 Project Overview 6
1.2 Objectives 6
Chapter 2 : Background 7
2.1 Problem Statement 7
2.2 Regulatory and Compliance Requirements 7
2.3 Existing solutions and their limitations 8
2.4 Current Challenges 9
Chapter 3 : Project Management 10
3.1 Overview of Development Methodology 10
3.2 Tools Used for Collaboration and Coordination 10
3.3 Communication Approach 11
3.4 The Communication Plan and Project Tracking 11
3.5 Stakeholder Engagement 12
3.6 Risk Assessment and Management 13
Chapter 4 : Development Approach 15
4.1 Overview of Development Methodology 15
4.2 Development Environment and Collaboration 16
4.2.1 Project Structure 17
4.3 Technology Stack 18
4.4 Development Stages 19
4.5 Testing Plan 21
4.5.1 Unit & Integration Testing 21
4.5.2 Security Testing 21
4.5.3 User Acceptance Testing (UAT) 22
4.5.4 File Integrity Verification Testing 22
Chapter 5 : Requirements 23
5.1 Introduction to Requirements 23
5.2 Functional Requirements 23
5.2.1 User Authentication and Role Management 23

5.2.2 File Upload and End-to-End Encryption 24
5.2.3 Workflow and Approval Chain 24
5.2.4 Audit Logging 25
5.2.5 File Download and Decryption 25
5.3 Non-Functional Requirements 26
5.3.1 Security 26
5.3.2 Usability 26
5.3.3 Performance 27
5.3.4 Reliability and Data Integrity 27
5.3.5 Compliance 27
5.4 System Modelling 27
5.4.1 Use Case Diagram 28
5.4.2 Entity Relationship Diagram (ERD) 30
5.4.3 Data Flow Diagram: File Upload Workflow 32
5.4.4 Sequence Diagram: Authentication Flow 34
Chapter 6 : Sprint Planning 36
6.1 Sprint Development Priority 36
6.2 Sprint Creation 36
6.3 Sprint Reviews and Team Reflections 37
Chapter 7 : Software Engineering / System Design 38
7.1 System Architecture 38
7.1.1 Frontend (Presentation Tier) 38
7.1.2 Backend (Logic Tier) 44
7.1.3 Database (Data Tier) 45
7.2 Security Design and Cryptography 45
7.2.1 End-to-End Encryption (E2EE) 45
7.2.2 Key Derivation (PBKDF2) 47
7.2.3 File Integrity Verification (SHA-256) 47
7.2.4 Zero-Knowledge Architecture 48
7.3 Role-Based Access Control and Workflow Management 49
7.3.1 User Roles 49

7.3.3 Three-Tier Approval Workflow 50
7.3.4 File Versioning 51
7.3.5 Deadline Management 51
7.4 Database Schema and Collections 52
7.4.1 Core Collections 52
7.4.2 Composite Indexes 53
7.4.3 Audit Trails and Accountability 54
7.5 Firestore Security Rules 55
7.5.1 Helper Functions 55
7.5.2 Key Security Rules 56
7.5.3 Firebase Storage Rules 56
7.6 Supported File Types 57
Chapter 8 : Quality Assurance 58
8.1 Overview 58
8.2 Testing Strategy 58
8.2.1 Unit Testing 58
8.2.2 Integration Testing 59
8.2.3 System and End-to-End Testing 60
8.2.4 Security Testing 60
8.3 Usability Testing 61
8.4 Bug Tracking and Resolution 62
8.5 Limitations and Reflections on QA 62
Chapter 9 : Conclusion 64
9.1 Summary 64
9.2 Technical and Practical Outcomes 64
9.3 Final Reflections 65
References 66
Appendices 68

Code Repository

Github	https://github.com/OoiWeiChyeh/MAL2020_SecureFileSharingWebApp_E2EE.git
Pebblepad	https://v3.pebblepad.co.uk/spa/#/public/mtw79ZbnmcwzfZhW78ZZbZjspW
Prototype	https://file-share-f8260.web.app/login
Presentation	https://www.youtube.com/watch?v=pZ3IU6vVbHs

Word Count : 10793 words

Chapter 1 : Introduction

1.1 Project Overview

This final report presents the development and implementation of a Secure File Share Management System, a web application designed to address the critical need for secure and auditable management of sensitive academic documents. Developed as part of the Computing Group Project (MAL2020), the system shows the practical integration of difficult cryptographic techniques with user-friendly workflow management for academic institutions.

1.2 Objectives

The primary objectives of this project are:

- **To Implement robust End-to-End Encryption (E2EE) using AES-256-GCM encryption** to ensure that files are encrypted at the client-side before transmission, with the server never accessing plaintext data.
- **To Establish a Three-Tier Approval Workflow** involving Lecturers, Heads of School (HOS), and Exam Unit administrators, ensuring comprehensive quality assurance and management oversight.
- **To Provide Complete Version Control and Audit Trails** to track all file modifications, approvals, and downloads, ensuring full accountability and compliance with data protection requirements.
- **To Implement Role-Based Access Control (RBAC)** with department-level isolation, ensuring data segregation and restricting access based on user roles and responsibilities.
- **To Demonstrate the feasibility of integrating strong cryptography** with practical, user-centric design to create a production-ready solution accessible to non-technical users.

Chapter 2 : Background

2.1 Problem Statement

Our institution faces challenges in managing exam papers securely throughout their lifecycle. Exam papers are highly sensitive documents that require multiple levels of quality assurance before printing, involving different users such as lecturers, heads of school and administrative staff. Traditional file management approaches lack adequate security measures to protect these documents from unauthorized access, ensure data integrity and maintain audit trails. Furthermore, existing solutions often fail to provide an intuitive workflow that balances security with usability, particularly for non-technical end-users. This project addresses these gaps by implementing a zero-knowledge architecture using end-to-end encryption (E2EE), ensuring that sensitive exam papers remain protected throughout their entire management lifecycle

2.2 Regulatory and Compliance Requirements

Universities must comply with various data protection regulations and institutional policies:

- **Data Protection Regulations**: GDPR and similar regulations require organizations to implement appropriate technical and organizational measures to protect personal and sensitive data.
- **Academic Integrity Standards**: Educational institutions have obligations to maintain the confidentiality and integrity of assessment materials to ensure fair and valid evaluations.
- **Institutional Policies**: Internal governance frameworks often mandate secure handling of sensitive academic documents with documented accountability measures.

2.3 Existing solutions and their limitations

While numerous generic file-sharing and document management solutions exist, they demonstrate notable limitations when applied to academic exam paper management:

Table 1 : Comparison table of existing solutions

Solution Type	Strengths	Limitations
Generic Cloud Storage (Google Drive, OneDrive)	User-friendly, widely adopted	Limited access granularity, no audit trails for exam workflows, encryption keys held by provider.
Enterprise File Systems (SharePoint, Alfresco)	Customizable, role-based access	Complex setup, expensive licensing, may not implement E2EE
Email and File Transfer	Familiar, widely used	Unencrypted, no version control, difficult tracking, security risks
General E2EE Solutions (Signal, ProtonMail)	Strong encryption, user control	Not designed for academic workflows, lack approval mechanisms, poor version control

None of these existing solutions adequately address the unique requirements of academic exam paper management, particularly the combination of:

- Strong end-to-end encryption where users maintain complete control of encryption keys

- Academic-specific workflow management with multi-level approval chains
- Comprehensive audit trails and version control tailored for examination processes
- User-friendly interfaces suitable for non-technical academic staff
- Department-level data isolation and access controls

2.4 Current Challenges

Traditional approaches to exam paper management within universities present several significant challenges:

Security Vulnerabilities: Conventional file-sharing platforms and email systems provide limited protection against unauthorized access. Papers often reside on unsecured network drives or are transmitted through unencrypted channels, exposing sensitive content to potential breaches.

Lack of Access Control: Most systems lack fine-grained access controls, making it difficult to ensure that only relevant stakeholders can view specific documents. Department-level data isolation is often inadequately implemented.

Insufficient Audit Trails: Traditional systems fail to provide comprehensive audit logs showing who accessed, modified, approved, or downloaded exam papers and when. This creates compliance and accountability gaps, particularly for regulatory requirements.

Version Control Limitations: Managing multiple versions of exam papers during the review and revision cycle is cumbersome and error prone. Tracking which version is current and maintaining previous versions for audit purposes is often manual and unreliable.

Workflow Inefficiencies: The approval process involving multiple stakeholders (lecturers, heads of school, exam units) lacks streamlined coordination, leading to delays and communication gaps. These challenges are consistent with findings documented in wider literature on secure document management in institutional settings; the OWASP Foundation (2021) similarly identifies insufficient access control and poor audit trail practices as systemic risks in information systems handling sensitive data.

Chapter 3 : Project Management

3.1 Overview of Development Methodology

This section explains how our team used the Agile methodology to manage the project's development lifecycle. We decided to manage the project as a collective effort, relying heavily on step-by-step development and getting regular feedback from our users. We broke our heavy workload into short, manageable time-boxed sprints that lasted between two to three weeks each. Working this way allowed us to focus completely on specific technical goals, such as building the user authentication, setting up the encryption modules, and managing the document workflow. Before starting a new sprint, we always started with a review of our previous workload. This allowed the team to discuss our progress and decide if we were happy with the completed work or if we needed to change our scope based on new ideas or testing results. To make sure we stayed on track, meetings were held regularly. We had both face-to-face physical meetings on campus and online virtual meetings through Zoom to plan our time and set clear expectations for the work that needed to be delivered.

3.2 Tools Used for Collaboration and Coordination

Having a clear communication plan was very important for the success of our team and to ensure the timely delivery of the secure file-sharing system. To make sure we worked together smoothly and kept accurate track of our project milestones, we utilized several different platforms. GitHub served as our primary version control system for managing the React.js and Node.js codebase. Using GitHub allowed our team to track every code change safely and code together at the same time without accidentally overwriting each other’s work; this distributed version control approach is a widely adopted practice in collaborative software development (Chacon and Straub, 2014). Before writing the code, we utilized Draw.io for system modelling and creating visual architecture diagrams. We used Microsoft Word for comprehensive report writing, documentation, and formatting our final project deliverables. Additionally, Zoom was very helpful for our online meetings and screen-sharing sessions, allowing us to troubleshoot difficult technical issues

collaboratively when we could not meet in person. For a visual record of our virtual coordination, please refer to the Zoom meeting screenshots provided in Appendix A.

3.3 Communication Approach

Making sure we had a proper communication plan was vital so that everyone in the group could work together smoothly. Our team operations ran smoothly because responsibilities were explicitly divided. We had a Team Leader coordinating the meetings and compiling deliverables, an Analyst defining what the users needed, a Designer creating the visual mock-ups, and a Programmer handling the system implementation. Every team member had a responsibility to finish their tasks on time. However, if someone was struggling, we made sure to help them so the whole team could move forward. For instance, the Programmer could not build the system if the Designer had not finished the layouts, so we had to communicate constantly to avoid delays. We had a strict rule that if someone was stuck on a technical issue like an encryption error, they should share it with the team immediately instead of keeping quiet. To avoid our documentation and development phases drifting out of sync, we shifted toward continuous check-ins rather than waiting until the end of a sprint to compile our work. Photographic documentation of our physical team meetings and sprint discussions can be found in Appendix B and Appendix C.

3.4 The Communication Plan and Project Tracking

Keeping the team aligned required a highly organized approach to managing our daily tasks and project files. For the development of the Kunchee web application, we established a centralized digital workspace to securely store all our technical documentation, meeting records, and design assets. To actively monitor our twenty-six-week development timeline, we relied on a dynamic sprint tracker. Our Team Leader took the responsibility of dividing the heavy development milestones into specific, manageable assignments. Every task within our sprint planner was given a strict deadline and clear instructions to ensure that we all knew exactly what they needed to deliver.

To maintain our momentum and check on everyone's progress, we held our main regular group meetings every Monday during our scheduled Computing Project class session. These weekly face-to-face sessions were crucial for reviewing our sprint tracker, updating

our completed tasks, and discussing our next steps. Because our development phases were deeply interconnected, we knew that a delay in one area could easily disrupt the entire schedule. Therefore, we established a strong team rule, if anyone faced a technical roadblock or struggled to meet a deadline, it was treated as a group problem. Instead of letting one person struggle alone, the entire team would brainstorm together, find a practical solution, and help each other complete the task. This collaborative support system ensured that difficult prerequisite tasks never held up the overall development pipeline.

Beyond managing our coding tasks and supporting each other, we maintained strict record-keeping to document our progress. For our weekly self-reflections, each team member wrote their own updates in a Microsoft Word document, noting their personal achievements and any technical challenges they faced. We then handed these files over to our Team Leader. As our representative, the Team Leader was the only person who accessed PebblePad to compile and upload all our project information, updates, and group logbooks. Furthermore, comprehensive meeting minutes were recorded after every single consultation with The Ship Campus Exam Unit. Keeping these detailed records ensured that every specific security request and privacy control was formally documented, providing the team with a reliable reference point whenever we needed to clarify requirements for the end-to-end encryption modules.

3.5 Stakeholder Engagement

Understanding the exact needs of the end-user is critical in Agile development. Because of this, our team held formal consultation meetings with our stakeholders, specifically the representatives from The Ship Campus Exam Unit. During these sessions, the team carefully recorded meeting minutes to lock in our core system requirements. These talks made sure we fully understood their rules and client expectations regarding data confidentiality, role-based access control, and end-to-end encryption compliance. Getting this feedback directly from the client helped us ensure that all these requirements were fully documented, understood, and integrated into our project goals. Detailed records of these discussions can be viewed in Appendix D, Appendix E, Appendix F, and Appendix

G which contain our official meeting minutes. For photographic evidence of our stakeholder meetings, please refer to Appendix H.

3.6 Risk Assessment and Management

Over the course of the project, we encountered several risks that could have slowed down our progress. Since exam papers are highly confidential, managing security risks was our focus. For technical risks, like the encryption failing silently, we made sure to use trusted tools like the Web Crypto API and libsodium.js. Unfamiliarity with these coding practices meant the team had to spend extra time learning, which could have caused development delays. We also faced issues running entirely online meetings, as some team members did not always have a reliable internet connection. This sometimes-caused members to miss parts of a meeting or lose the chance to voice their concerns. To combat this, we planned our Zoom meetings to be longer than expected to allow time for people to reconnect and catch up. For operational mistakes, like a user accidentally uploading the wrong exam paper, we added audit logs and version tracking so we can easily trace errors. Lastly, we cannot stop users from falling for phishing scams, but we used role-based access to limit how much damage a stolen account can do.

To illustrate the risks, and calculate their probability and impact, a risk matrix can be used like so:

Table 2 : Risk Matrix table

	Impact							
Probability	Risk Matrix	1	2	3	4	5		Key
	1	1	2	3	4	5		Low
	2	2	4	6	8	10		Medium
	3	3	6	9	12	15		High
	4	4	8	12	16	20		Very High
	5	5	10	15	20	25		

While we can only roughly estimate the probability of each risk as usually past projects are used as the basis of such estimations, we can certainly consider the impact. Using the risk matrix, we can calculate a risk score for each scenario by multiplying its likelihood by the impact. By visualizing the risk assessment, the following table can be designed:

Table 3 : Risk Assessment table

Risk	Score	Level
Users falling victim to external phishing attacks.	12	High
Incorrect implementation of end-to-end encryption.	10	High
Users accidentally uploading the wrong version of an exam paper.	6	Medium
Development delays due to unfamiliarity with React.js or Node.js.	6	Medium
Corrupted or lost cryptographic keys causing data loss.	5	Medium

Chapter 4 : Development Approach

4.1 Overview of Development Methodology

For the development of the Kunchee secure file-sharing system, the team utilized the Agile development methodology. This approach allowed for iterative development, enabling the team to break down the heavy workload into short, manageable, time-boxed sprints lasting between two to three weeks each. Agile was chosen over a traditional Waterfall model because the security requirements for this system evolved through stakeholder consultation; iterative sprints allowed requirements to be refined at each cycle rather than being locked in during an upfront specification phase, which would have been unsuitable given the evolving nature of the encryption and workflow requirements (Beck et al., 2001; Sommerville, 2016). Working in sprints allowed the team to focus on specific technical objectives such as building user authentication, implementing end-to-end encryption modules, designing role-based approval workflows, and configuring the database security rules while gathering user feedback and adapting to changing requirements.

The Agile approach proved especially effective given the security-sensitive nature of the project. Each sprint delivered a working increment of the system that could be tested for both functional correctness and security compliance. For instance, the team was able to implement the encryption layer in one sprint and then validate it against OWASP security standards in the next, before moving on to the approval workflow features. This iterative cycle of build-test-refine ensured that each layer of the system was stable before introducing new features on top of it.

4.2 Development Environment and Collaboration

To ensure smooth coordination and safe code integration, the team relied on a robust suite of development tools and technologies:

- **GitHub** served as the primary version control system to manage the React.js and Node.js codebase, allowing multiple team members to track code changes across branches, review pull requests, and collaborate without overwriting each other's work.
- **Visual Studio Code (VS Code)** was utilized as the main Integrated Development Environment (IDE) for coding, debugging, and running the Firebase emulator suite during local development.
- **Draw.io** was used for system modeling and architecture visualization prior to coding, including creating the system architecture diagram, Entity-Relationship Diagram (ERD), and approval workflow flowchart.
- **Firebase Emulator Suite** was utilised extensively during development for local testing. The emulator provided simulated instances of Firebase Authentication (port 9099), Cloud Firestore (port 8080), Firebase Storage (port 9199), and Cloud Functions (port 5001), reachable via the Emulator UI at localhost:4000. This allowed the team to develop and test all features locally without incurring costs or risking corruption of the production database.
- **Vite** was used as the frontend build tool and development server, providing near-instant Hot Module Replacement (HMR) during development. Vite's fast cold-start and on-demand compilation significantly improved developer productivity compared to traditional bundlers.

4.2.1 Project Structure

The codebase was organized using a modular folder structure to promote separation of concerns and maintainability:

Table 4 : File Structure Table

Folder	Purpose
src/pages/	10 page-level React components (Login, Register, Dashboard, Upload, ViewFile, Settings, AdminPanel, HOSReview, ExamUnitReview, PendingApproval)
src/components/	15 reusable UI components (Navbar, FileCard, PDFViewer, NotificationsPanel, VersionHistoryModal, etc.)
src/services/	6 service modules handling business logic (authService, encryptionService, storageService, firestoreService, notificationService, emailService)
src/utils/	Shared constants and helper functions

This modular architecture ensured that the encryption logic, Firebase interactions, and UI rendering remained decoupled, making the system easier to debug and extend.

4.3 Technology Stack

The following table summarizes the full technology stack used in the Kunchee system:

Table 5 : Technology Used in the Project

Layer	Technology	Version	Purpose
Frontend Framework	React.js	18.2.0	Dynamic, component-based user interface
Routing	React Router DOM	6.20.0	Client-side page navigation and route protection
Build Tool	Vite	5.0.8	Fast development server and production bundler
CSS Framework	Tailwind CSS	3.3.6	Utility-first responsive styling
Icons	Lucide React	0.294.0	Consistent icon set across the UI
Authentication	Firebase Authentication	10.14.1	Email/password and Google OAuth sign-in
Database	Cloud Firestore	10.14.1	NoSQL cloud database for metadata, user profiles, and audit logs
File Storage	Firebase Storage	10.14.1	Cloud storage for encrypted file blobs
Serverless Functions	Firebase Cloud Functions	—	Backend logic (email notifications)
Encryption	Web Crypto API (browser-native)	—	AES-256-GCM file encryption and PBKDF2 key derivation
File Integrity	Web Crypto API - SHA-256	—	Hash generation and verification for tamper detection

Spreadsheet Parsing	SheetJS (xlsx)	0.18.5	Parsing Excel files for batch user/department import
File Download	FileSaver.js	2.0.5	Triggering browser file download after decryption

4.4 Development Stages

The development lifecycle was structured into several key stages:

Stage 1 — Requirements Gathering & System Design

Defining what the Exam Unit needed and creating architectural diagrams and database structures using Draw.io. This included mapping out the university's existing paper-based approval hierarchy (Lecturer → HOS → Exam Unit) and translating it into a digital workflow. The Firestore database schema was designed at this stage, identifying the core collections required: users, files, fileVersions, feedback, notifications, departments, and auditLogs.

Stage 2 — Authentication & User Management

Implementing Firebase Authentication with two sign-in methods: **email/password registration** and **Google OAuth (Sign-in with Google)**. A user whitelist (pre-registration) system was built to allow the Exam Unit to pre-approve users via Excel spreadsheet import. When a pre-registered user signs up, the system automatically assigns their role and department without requiring manual approval, streamlining the onboarding process.

Stage 3 — Frontend Development (Phase 1)

Developing the basic User Interface with React.js, including the Login, Registration, Dashboard, Upload, and Settings pages. React Router DOM was configured with three layers of route protection:

- ProtectedRoute — ensures user is authenticated
- PublicRoute — redirects authenticated users away from login/register pages

- RoleProtectedRoute — checks both authentication AND user role (e.g., only exam_unit can access /admin)

Stage 4 — Backend & CRUD Operations (Phase 2)

Establishing the Firestore data layer with a comprehensive service module (firestoreService.js over 2,100 lines) handling 90+ functions across 12 functional areas: file metadata CRUD, role management, department/course/subject management, file versioning, feedback, notifications, workflow status transitions, audit logging, pre-registered user whitelist, batch import, and deadline management.

Stage 5 — Encryption Integration

Integrating the **Web Crypto API** to ensure files were encrypted client-side before transmission. The encryption module (encryptionService.js) implements:

- **AES-256-GCM** symmetric encryption with a 12-byte Initialization Vector (IV) for file content
- **PBKDF2** (Password-Based Key Derivation Function 2) with 100,000 iterations and SHA-256 hashing for deriving encryption keys from passwords
- **SHA-256** hash generation at upload time and hash verification at download time for file integrity assurance
- A per-file unique encryption key generation, ensuring each uploaded file has its own independent AES-256 key

Stage 6 — Approval Workflow Implementation

Building the three-tier approval workflow engine with status transitions (DRAFT → PENDING_REVIEW → HOS_APPROVED → EXAM_UNIT_APPROVED) and rejection/revision flows. This included constructing the HOSReview and ExamUnitReview pages, feedback creation with optional file attachments, and automatic notification dispatching to relevant users at each status change.

Stage 7 — Testing & Refinement

Fixing interface navigation bugs and resolving workflow issues prior to the final handover. This included debugging Firestore security rule conflicts, resolving merge conflicts in critical files, and validating that the encryption/decryption cycle worked correctly across all supported file types.

4.5 Testing Plan

Quality assurance was embedded throughout the development lifecycle to ensure system stability and security. The testing approach included:

4.5.1 Unit & Integration Testing

Utilizing **Jest** and **Mocha** to verify individual features (such as login, upload, and download functions) and ensure all backend and frontend components communicated seamlessly. The Firebase Emulator Suite was used to run integration tests locally against emulated Firestore data without requiring a network connection or production credentials.

4.5.2 Security Testing

Applying **OWASP ZAP** (Zed Attack Proxy) to identify potential vulnerabilities and verify that the end-to-end encryption (E2EE) implementation was robust and secure against interception. A dedicated security audit was conducted that identified and resolved issues including:

- Hardened Firestore security rules preventing unauthorized database queries
- Implementation of role self-promotion prevention (users cannot change their own role field)
- Protection of encryption keys in share links using URL fragments (#key=) instead of query parameters, since fragments are never sent to the server
- Firebase Storage rules enforcing a 50 MB file size limit and ownership-based write access

4.5.3 User Acceptance Testing (UAT)

Gathering feedback directly from the stakeholders, including Lecturers, Heads of School (HOS), and Exam Unit staff, to ensure the UI remained simple and intuitive. Feedback from UAT sessions led to improvements such as adding the file category selection system (Question Paper, Answer Key, Rubric, Marking Scheme, Model Answer, Other) and the submission deadline tracking feature with cascading deadlines (subject → course → department levels).

4.5.4 File Integrity Verification Testing

The SHA-256 hash-based integrity verification system was tested by:

1. Uploading a file and capturing the generated hash stored in Firestore
2. Downloading and decrypting the same file
3. Regenerating the SHA-256 hash of the decrypted output
4. Comparing both hashes to confirm they match (integrity verified)
5. Testing with a deliberately corrupted file to confirm that hash mismatches are correctly flagged to the user

Chapter 5 : Requirements

5.1 Introduction to Requirements

Before writing a single line of code, the team spent time trying to understand what the system actually needed to do and perhaps more importantly, what it did not need to do. In projects like this, it is tempting to add features because they sound useful, but keeping the scope tight is what makes or breaks a student project. The requirements for Kunchee were gathered through two stakeholder meetings with the Examination Unit at The Ship Campus, informal team discussions and a review of how similar tools like Signal and Wormhole handle secure file transfer.

Requirements are generally divided into two broad categories: functional and non-functional. Functional requirements describe what the system should do. Non-functional requirements capture how well it should do those things, covering aspects like performance, security, and usability. Both matter, though in practice the non-functional ones are easier to overlook during development and harder to retrofit later.

5.2 Functional Requirements

The functional requirements for Kunchee were mapped to the core workflow it is meant to support: a lecturer creates an exam paper, passes it up through the Head of School for approval, and it eventually reaches the Examination Unit for printing. Every feature was evaluated against whether it actually served that workflow or was just a nice-to-have.

5.2.1 User Authentication and Role Management

The system requires a secure login mechanism for all users. Given the sensitivity of exam content, basic username-and-password login alone was considered insufficient. Kunchee therefore implements two-factor authentication (2FA) as a mandatory step at login. This is worth emphasising because many systems offer 2FA but leave it as a user choice, which means in practice most people never enable it.

Role-based access control (RBAC) sits at the core of how the system restricts visibility. Three roles are defined:

- Lecturer — can upload exam papers and view only their own submissions
- Head of School (HOS) — can review and approve submissions from lecturers within their school
- Examination Unit Administrator — has the highest level of access and manages the final distribution workflow

Each role carries different permissions and the system enforces these on both the client and server side. Relying only on client-side enforcement would cause a user could simply bypass it through the browser's developer tools.

5.2.2 File Upload and End-to-End Encryption

Users need to be able to upload exam papers in common file formats. Before any file leaves the user's browser, it is encrypted on the client side using AES-GCM via the Web Crypto API. This means the server never receives a readable version of the file. Even if someone with database access tried to open a stored file, they would see nothing useful.

The key point here is that encryption happens before upload, not during or after. This is the defining characteristic of end-to-end encryption: the server is effectively blind to the content it is storing. It is worth noting that this approach does introduce some trade-offs. For instance, server-side search across file contents becomes impossible, and if a user loses their encryption key, the file is unrecoverable. For an exam paper system, exam papers do not need to be searchable by content and key management can be handled through the existing user account structure.

5.2.3 Workflow and Approval Chain

A significant part of what Kunchee does is manage the sequential approval process. Once a lecturer uploads a paper, it enters a workflow state of 'Pending Review'. The Head of School can then either approve or return it with comments. Upon HOS approval, it moves

to the Examination Unit queue. This chain maps directly to what the stakeholders described during the October 2025 meeting.

The system should also prevent a paper from being accessible by the next stage in the chain until the current stage has explicitly approved it. This kind of gating is not glamorous from a development standpoint, but it is what makes the workflow meaningful rather than just decorative.

5.2.4 Audit Logging

Every meaningful action in the system like file upload, approval, download, login should be logged with a timestamp and the identity of the user who performed it. The log entries are read-only; no user, including admins, should be able to delete or modify them. This creates a chain of custody for exam papers which was identified as a key gap in the existing USB-based process.

Audit logs are not the most exciting feature to implement, but they matter enormously in practice. If an exam paper were ever alleged to have leaked, the logs would be the first place anyone looked. Having no logs is arguably worse than having no encryption, because at least with no encryption you know you have a problem like without logs, you may never find out how it happened.

5.2.5 File Download and Decryption

Authorised users should be able to download and decrypt files that have been shared with them. Decryption happens client-side, mirroring the encryption process. The user receives the ciphertext from the server, and their browser decrypts it locally before presenting the file. The server at no point handles the plaintext.

5.3 Non-Functional Requirements

Non-functional requirements tend to get less attention in student project reports, perhaps because they are harder to demonstrate in a demo. That said, they are often what determines whether a real-world deployment succeeds or fails.

5.3.1 Security

Beyond the functional encryption requirements already described, the system should follow OWASP's Top 10 guidelines as a baseline. This includes protections against common vulnerabilities like cross-site scripting (XSS), SQL injection (though Firebase's structure largely sidesteps traditional SQL concerns), and insecure direct object references. The use of Firebase's built-in security rules helps enforce these at the database layer, but the application code itself still needs to be written carefully.

All communication between the client and server must occur over HTTPS. This may seem obvious, but it is worth stating explicitly because development environments sometimes run over HTTP, and habits formed during development can sometimes carry over into deployment if the team is not careful.

5.3.2 Usability

The system is intended to be used by lecturers and administrative staff, many of whom are not particularly technical. The interface needs to be straightforward enough that someone who has never seen it before can complete the upload workflow without a manual. This is a harder constraint than it sounds. It requires resisting the urge to expose complexity in the UI just because the system is complex under the hood.

During stakeholder discussions, it became clear that one of the main complaints about the current USB-based process is that it feels fragile and error-prone. The new system should feel reliable. Error messages should be human-readable, not stack traces. Actions should feel reversible where possible and the system should confirm when something has worked rather than staying silent.

5.3.3 Performance

The system does not need to handle thousands of concurrent users. The Examination Unit at The Ship Campus is relatively small and exam papers are uploaded in batches at specific points in the academic calendar rather than continuously. That said, the encryption and decryption operations should not make file transfers feel slow. In testing, AES-GCM encryption of typical exam-paper-sized PDFs (usually under 5MB) ran in well under a second on standard hardware which is acceptable.

5.3.4 Reliability and Data Integrity

Files should arrive at their destination intact and unmodified. The system should detect and reject corrupted uploads rather than silently storing a broken file. Firebase's storage layer provides some built-in redundancy, but the application should still verify checksums or use the authentication tag built into AES-GCM to confirm decryption succeeded.

5.3.5 Compliance

Given the Malaysian context, the system design should broadly align with the Personal Data Protection Act 2010 (PDPA). In practice, this primarily means minimising the personal data collected (names and institutional email addresses, nothing more), restricting access to that data based on role, and ensuring sensitive content is stored encrypted. The system does not collect payment information or other categories of sensitive personal data, which simplifies compliance somewhat.

5.4 System Modelling

To support both development and documentation, the team produced several models capturing different aspects of the system. These are not merely academic exercises. They were genuinely useful during implementation for catching ambiguities before they became bugs.

5.4.1 Use Case Diagram

Figure 1 : Use Case Diagram

The Use Case Diagram (Figure 1) models the functional boundaries of the Kunchee system from the perspective of its two primary actors: User (which encompasses Lecturer, Head of School, and Exam Unit roles depending on the context) and Exam Unit (in its administrative capacity). The diagram uses standard UML notation with «include» and «extend» relationships to show mandatory and optional behaviour chains respectively.

Key observations from the diagram include: (1) Login to System «include» Two-Factor Authentication, meaning 2FA is a mandatory sub-process that cannot be bypassed. This was a deliberate security design decision supported by OWASP Foundation (2021) guidance on authentication best practices. (2) Create Exam Paper «include» Encrypt Paper Data, confirming that encryption is an inseparable part of the file creation workflow rather than an optional feature. (3) Review Exam Paper «extend» Request Revision and «extend» Approve Exam Paper, showing that approval and rejection are conditional extensions of the review action rather than guaranteed outcomes. (4) The Exam Unit actor exclusively controls Manage User Roles, Audit Activity Logs, and Access All Papers, which are the three administrative capabilities outside the standard user workflow. The diagram helped clarify a critical boundary during design: Heads of School do not appear as a separate named actor because their behaviour is captured through the User actor with role-specific access control enforced at the application layer, consistent with the RBAC model described by Sandhu et al. (1996).

5.4.2 Entity Relationship Diagram (ERD)

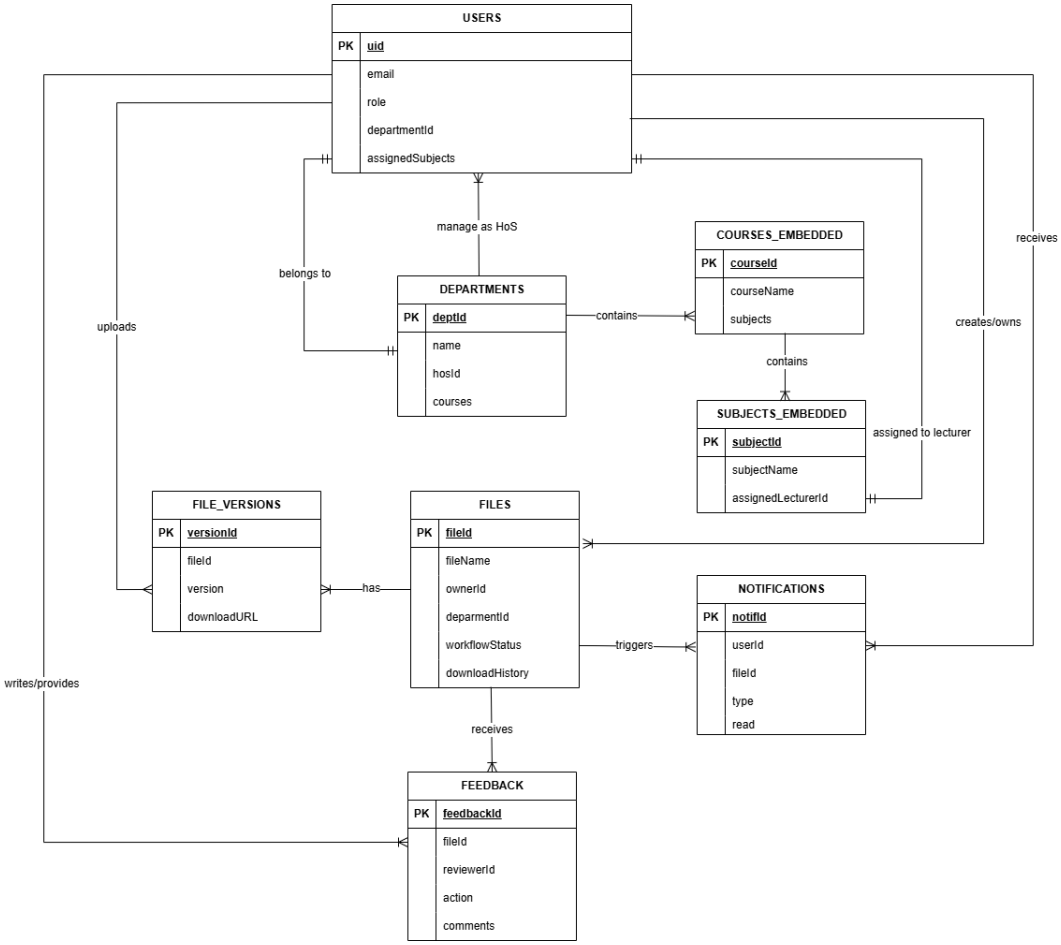

Figure 2 : Entity Relationship Diagram

The Entity Relationship Diagram (Figure 2) models the data schema underlying the Kunchee system using standard crow's foot notation to express cardinality between entities. The diagram captures eight core entities: USERS, DEPARTMENTS, COURSES_EMBEDDED, SUBJECTS_EMBEDDED, FILES, FILE_VERSIONS, NOTIFICATIONS, and FEEDBACK. The key relationships are as follows. A USERS entity holds the uid (primary key), email, role, departmentId, and assignedSubjects fields; each user belongs to exactly one DEPARTMENT and may upload zero or more FILES. DEPARTMENTS contain COURSES_EMBEDDED (one-to-many), which in turn contain SUBJECTS_EMBEDDED (one-to-many) — this nested hierarchy reflects the real-world academic structure and allows deadline rules to cascade from department level down to individual subject level. FILES are the central entity: each file references its ownerId,

departmentId, workflowStatus, and downloadHistory. A single file may have multiple FILE_VERSIONS (one-to-many), each with its own versionId, encryptionKey, and downloadURL, enabling full revision history. Each file may trigger zero or more NOTIFICATIONS directed to specific users, and may receive zero or more FEEDBACK records written by reviewers during the approval workflow.

A notable design decision captured in the ERD is the presence of an encryptionKey field within both FILES and FILE_VERSIONS. Unlike many document management systems where encryption keys are managed entirely outside the database, Kunchee stores per-file AES-256 keys in Firestore protected by role-based security rules. This was a deliberate trade-off: storing keys in Firestore enables authorised multi-user access to shared files without requiring out-of-band key exchange, at the cost of the server having technical access to keys. As discussed further in Section 7.2.4, this model is more accurately described as access-controlled key storage rather than strict zero-knowledge architecture. The ERD also confirms that USERS never stores raw passwords. Authentication is delegated entirely to Firebase Authentication, so the USERS collection contains only profile and role metadata.

5.4.3 Data Flow Diagram: File Upload Workflow

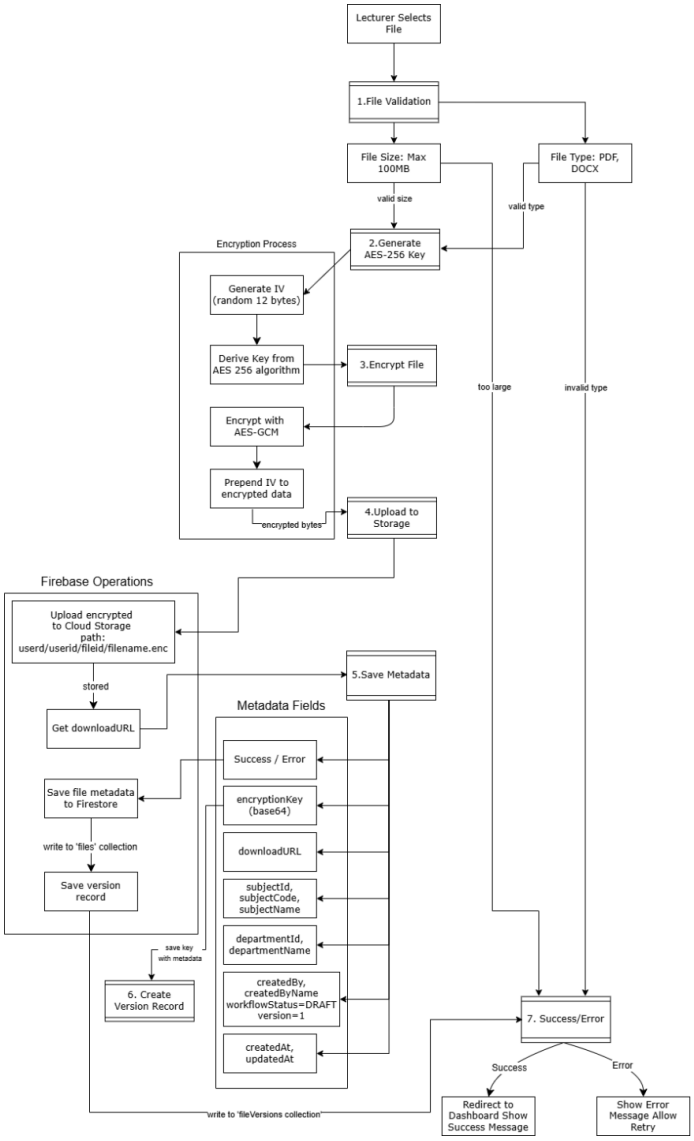

Figure 3 : Data Flow Diagram

The Data Flow Diagram (Figure 3) traces the complete file upload workflow from the moment a Lecturer selects a file to the point at which a version record is committed in Firestore. The diagram is divided into two swimlane regions such as client-side (Encryption Process) and server-side (Firebase Operations), making explicit where each processing step occurs and therefore where the system's security boundary lies.

The seven numbered steps in the diagram are:

(1) File Validation where the client checks file size (max 100 MB) and MIME type (PDF, DOCX, etc.) before any processing begins; invalid files are rejected immediately without network activity.

(2) Generate AES-256 Key — a unique 256-bit AES key is generated via **crypto.subtle.generateKey()** in the browser; this step is entirely client-side and the raw key material never leaves the browser in plaintext (W3C, 2023).

(3) Encrypt File — the Encryption Process sub-flow generates a random 12-byte Initialization Vector (IV), derives the key using the AES-256 algorithm, encrypts the file bytes using AES-GCM mode (NIST, 2007), and prepends the IV to the ciphertext to produce a single encrypted byte array.

(4) Upload to Storage — only the encrypted blob (with .enc extension and application/octet-stream content type) is transmitted to Firebase Storage via the client SDK; the server receives ciphertext only.

(5) Save Metadata — the system writes structured metadata fields to the Firestore 'files' collection including the base64-encoded encryptionKey, downloadURL, subjectId, departmentId, workflowStatus (set to DRAFT), and version number (set to 1).

(6) Create Version Record — a parallel record is created in the 'fileVersions' collection, saving the encryption key alongside version-specific fields to support the multi-version history feature.

(7) Success/Error — on successful completion the user is redirected to the Dashboard with a confirmation message; on failure an error message is shown and the user may retry. Critically, the diagram confirms that the encryption key is saved with metadata (step 5) only after the upload to Firebase Storage completes (step 4), ensuring no orphaned key records exist in the database for failed uploads.

5.4.4 Sequence Diagram: Authentication Flow

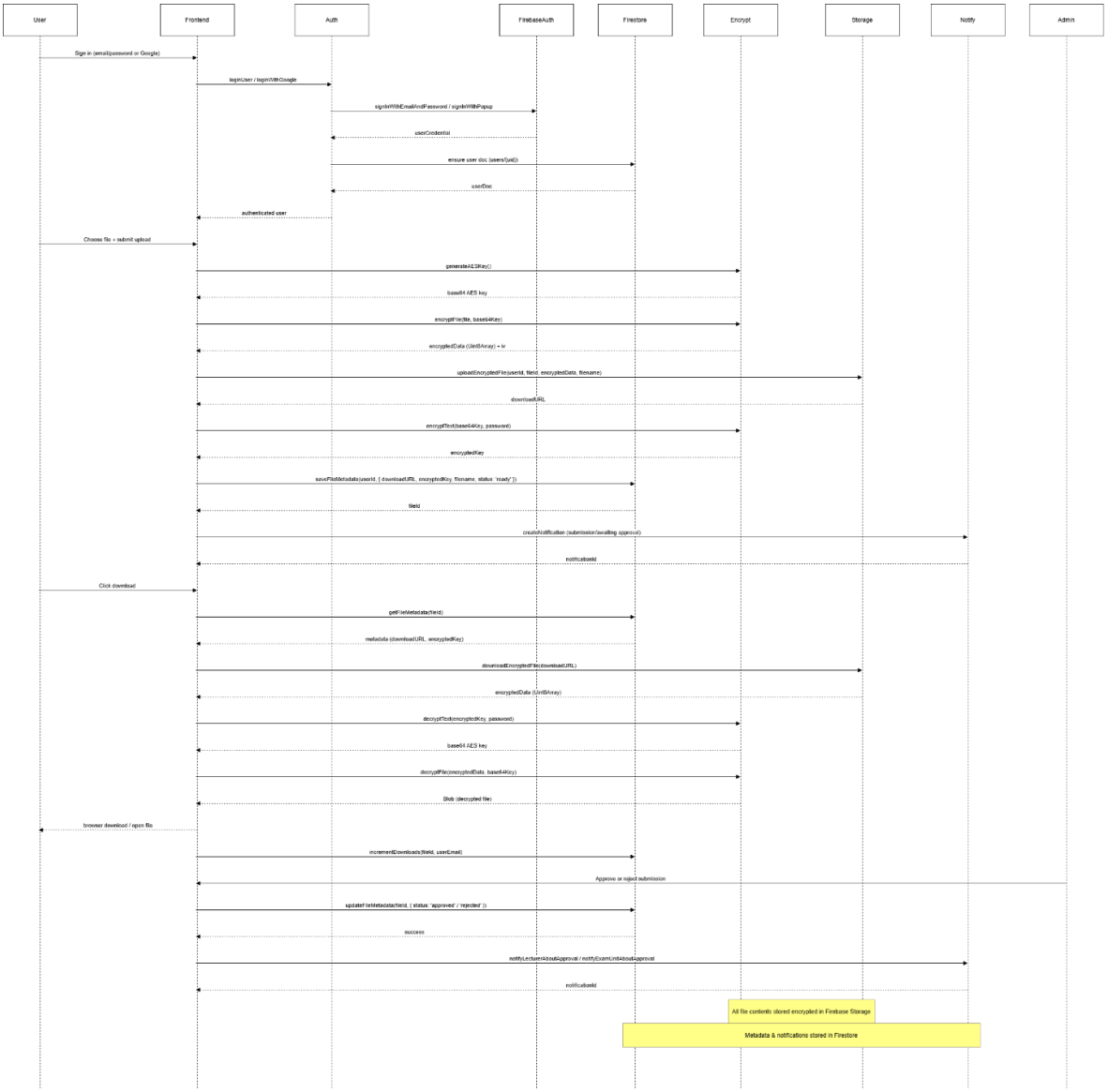

Figure 4 : Sequence Diagram

The Sequence Diagram (Figure 4) depicts the full interaction lifecycle of the Kunchee system across nine lifelines: User, Frontend, Auth, FirebaseAuth, Firestore, Encrypt, Storage, Notify, and Admin. The diagram covers three major workflows in sequence: user authentication and file upload, file download and decryption and approval/rejection by a reviewer.

In the authentication and upload flow: the User submits credentials to the Frontend, which calls `loginUser()` or `loginWithGoogle()` on the Auth service. Auth invokes `signInWithEmailAndPassword()` or `signInWithPopup()` on FirebaseAuth, which returns a userCredential. Auth then calls ensure user doc (users/{uid}) on Firestore to retrieve the user's role and profile, returning a userDoc. The authenticated user state is then propagated back to the Frontend. When the user chooses a file and submits an upload, the Frontend calls `generateAESKey()` on the Encrypt service (entirely client-side), receives the base64 AES key, then calls `encryptFile(file, base64Key)` to produce `encryptedData (Uint8Array)` and the `IV`. The Frontend then calls `uploadEncryptedFile(userId, fileId, encryptedData, filename)` on the Storage service, which returns a downloadURL. Next, `encryptToB64(base64Key, password)` is called on Encrypt to produce the encryptedKey for storage, and `saveFileMetadata(userId, downloadURL, encryptedKey, filename, status: 'ready')` is called on Firestore, returning the fileId. Finally, `createNotification (submission/awaiting approval)` is sent to the Notify service.

In the download and decryption flow: the User clicks download, triggering `getFileMetadata(fileId)` from Firestore, which returns metadata including the downloadURL and encryptedKey. The Frontend then calls `downloadEncryptedFile(downloadURL)` on Storage, receiving `encryptedData (Uint8Array)`. It calls `decryptToB64(encryptedKey, password)` on Encrypt to recover the base64 AES key, `then decryptFile(encryptedData, base64Key)` to produce the decrypted Blob. The browser download or file open event is triggered on the user's device, `and incrementDownloads(fileId, userEmail)` is called to update the audit record in Firestore. In the approval flow, the Admin (HOS or Exam Unit) triggers `updateFileMetadata(fileId, {status: 'approved'/'rejected'})` on Firestore, which returns success, and `notifyLecturerAboutApproval` or `notifyExamUnitAboutApproval` is dispatched via Notify. The diagram also includes two summary notes confirming the security boundary: all file contents are stored

encrypted in Firebase Storage, and metadata and notifications are stored in Firestore only that consistent with the client-side encryption architecture described in Section 7.2.

Chapter 6 : Sprint Planning

6.1 Sprint Development Priority

At the start of the project, our team carefully laid out the core objectives based on the requirements gathered from the Examination Unit. We broke these large requirements down and allocated them to specific sprints. Because our secure file-sharing system focuses heavily on data confidentiality, we had to plan our development order very strictly. For example, we knew that the basic React.js user interface and the Firebase authentication endpoints had to be finished before we could successfully integrate and test the complex end-to-end encryption modules. Our main priority for the first half of the timeline was to establish a solid structural foundation and a working frontend prototype for our interim submission. Once this baseline was secured, our focus for the remaining sprints shifted entirely toward integrating the Web Crypto API, refining the role-based access controls, and rigorously testing the secure document workflow before the official handover.

6.2 Sprint Creation

We planned our entire project schedule using a comprehensive Gantt chart that covered a twenty-six-week timeline. This schedule started in late September 2025 and continued until our final system submission in April 2026. To make this massive workload easier to handle, we divided the development phase into Agile sprints that lasted between two to three weeks each. The early sprints were dedicated to requirement gathering and system architecture, while the middle sprints focused heavily on coding the Kunchee application. We tracked our sprint progress and task assignments using our GitHub project board. Additionally, we utilized our regular physical meetings on campus during our Monday Computing Project class, as well as online Zoom sessions, to ensure everyone was on the same page. These meetings gave everyone a clear chance to talk about any technical

problems they were facing and allowed us to adjust our Gantt chart if certain coding tasks took longer than expected.

6.3 Sprint Reviews and Team Reflections

A big part of our Agile method was checking our progress at the end of every sprint. We used our regular team meetings to do both sprint reviews and team reflections. In the sprint reviews, we focused on the application itself. We talked about what we managed to finish, what tasks were late, and what bugs needed fixing before starting the next sprint. After that, we did our team reflections, which were all about our teamwork and communication. In these sessions, we discussed how the group was doing and how we could work together better. For example, if poor communication was slowing down our coding, we used this time to figure out a solution and change our plan for the next sprint. This was a helpful habit because it gave us a chance to look back, fix our mistakes, and improve how we worked as a team throughout the whole twenty-six weeks.

Week	Task / Milestone	Expected Output	Start Date	End Date
1	Requirement Gathering	Problem Statement, Objectives, User Requirements	29/09/2025	03/10/2025
2	Literature Review	Literature Review Summary	06/10/2025	10/10/2025
3	UI/UX Design (Figma)	Initial Mockups & User Flow	13/10/2025	17/10/2025
4	Client Preparation (Slides + Questionnaire)	Consultation Materials	20/10/2025	24/10/2025
5	System Architecture & Database Planning	Architecture Diagram + ERD	27/10/2025	31/10/2025
	Development Phase 1 Start	Project Setup + DB Setup		
6	Frontend Development (Phase 1)	Basic UI Screens	03/11/2025	07/11/2025
7	Backend Development (Phase 1)	Authentication + CRUD	10/11/2025	14/11/2025
	Encryption Module (Phase 1)	Basic Encryption Prototype		
8	Integration & Debugging	Integrated System v1	17/11/2025	21/11/2025
9	Prototype Improvement	Improved UI & Flow	24/11/2025	28/11/2025
10	Interim video + Report	Submission on 05/12/2025	01/12/2025	05/12/2025
11	Frontend Development (Phase 2)	Additional UI Screens	08/12/2025	12/12/2025
12	Backend Development (Phase 2)	Additional Backend Features	15/12/2025	19/12/2025
13	SEMESTER BREAK		22/12/2025	11/01/2026
14				
15	System Testing (Phase 1)	Bug Report & Fixes	12/01/2026	16/01/2026
16	User Feedback Collection	Feedback Report	19/01/2026	23/01/2026
17	System Refinement	Improved Stability	26/01/2026	30/01/2026
18	Frontend Polish & UX Update	Finalized UI Design	02/02/2026	06/02/2026
19	Backend Optimisation	Faster & Stable Backend	09/02/2026	13/02/2026
20	Security Review (E2EE Focus)	Security Checklist	16/02/2026	20/02/2026
21	System Testing (Phase 2)	Final test report	23/02/2026	27/02/2026
22	Final Prototype Build	Prototype v2 (Final)	02/03/2026	06/03/2026
23	Video Preparation	Prototype Demo Video	09/03/2026	13/03/2026
24	Final Presentation Preparation	Presentation Slides	16/03/2026	20/03/2026
25	Final Documentation	Documentation	23/03/2026	27/03/2026
26	Submission Week (4 April 2026)	Final Report & Prototype	30/03/2026	01/04/2026

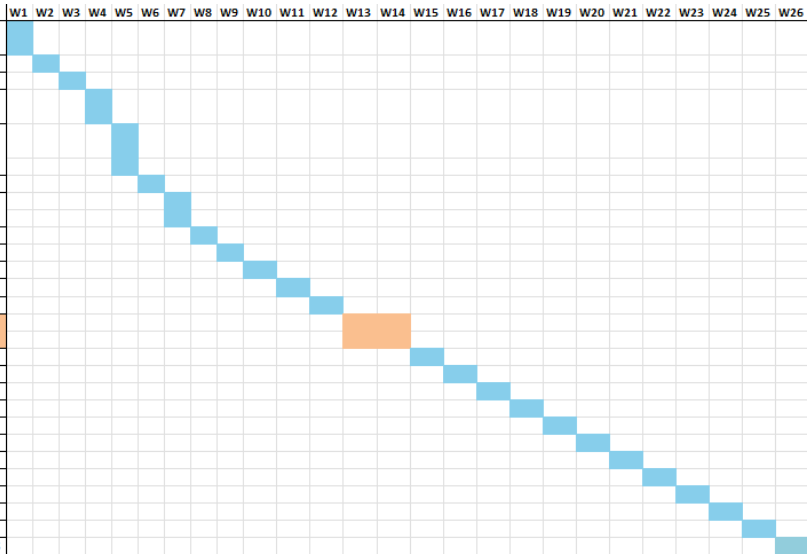

Figure 5 : Project Gantt Chart

Chapter 7 : Software Engineering / System Design

7.1 System Architecture

The Kunchee application follows a modern, **three-tier web architecture** built for scalability and responsiveness:

7.1.1 Frontend (Presentation Tier)

Built using **React.js 18.2** with **Vite 5.0** as the build tool. The frontend provides a dynamic and responsive Single Page Application (SPA) that adapts to various screen sizes and browsers. The UI is styled using **Tailwind CSS 3.3** for utility-first responsive design and uses **Lucide React** for a consistent icon library. The frontend is structured as follows:

- **10 Page Components**:

  Login, Register, Dashboard, Upload, ViewFile, Settings, AdminPanel, HOSReview, E xamUnitReview, PendingApproval

- **15 Reusable Components**:

  Including Navbar, FileCard, ReviewFileCard, PDFViewer, NotificationsPanel, Version HistoryModal, FileTimelineModal, UploadNewVersionModal, BatchImportTab, Deadli neSettingsTab, HOSUserManagement, ConfirmModal, FeedbackFileInput, Skeleton Card, ToastProvider

- **Client-Side Route Protection**:

  Three layers of route guards: ProtectedRoute (authentication), PublicRoute (redirect if logged in), and RoleProtectedRoute (role-based access)

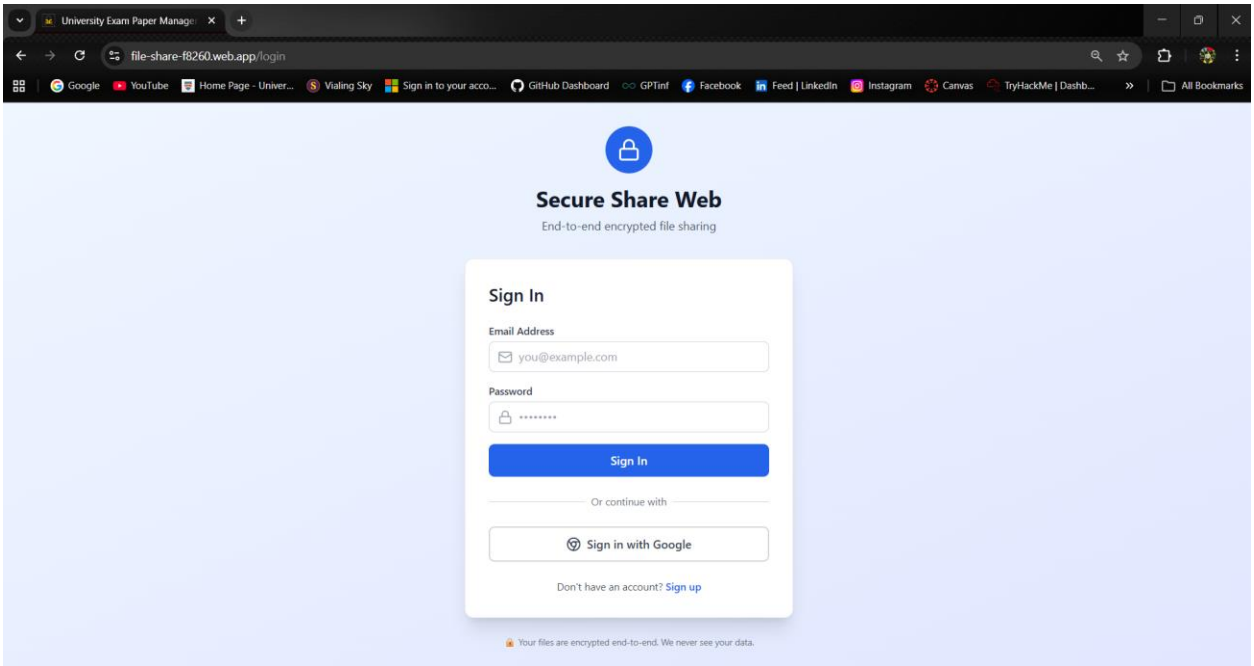

Figure 6 : User Login Page

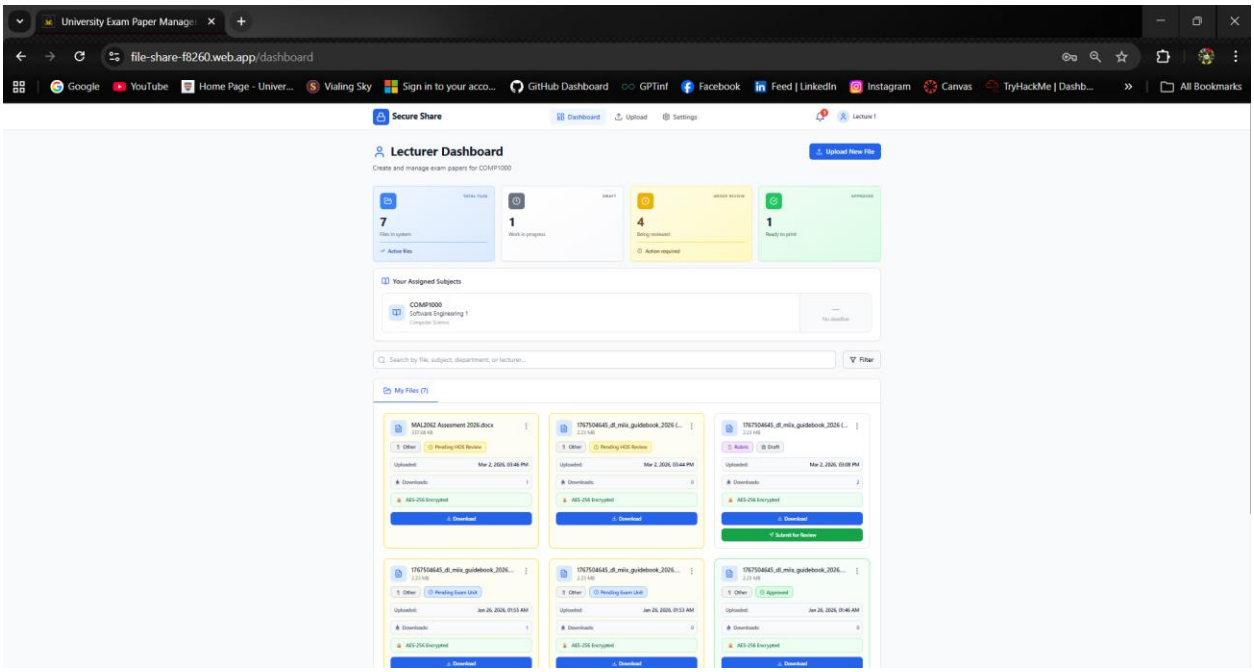

Figure 7 : Lecturer Page

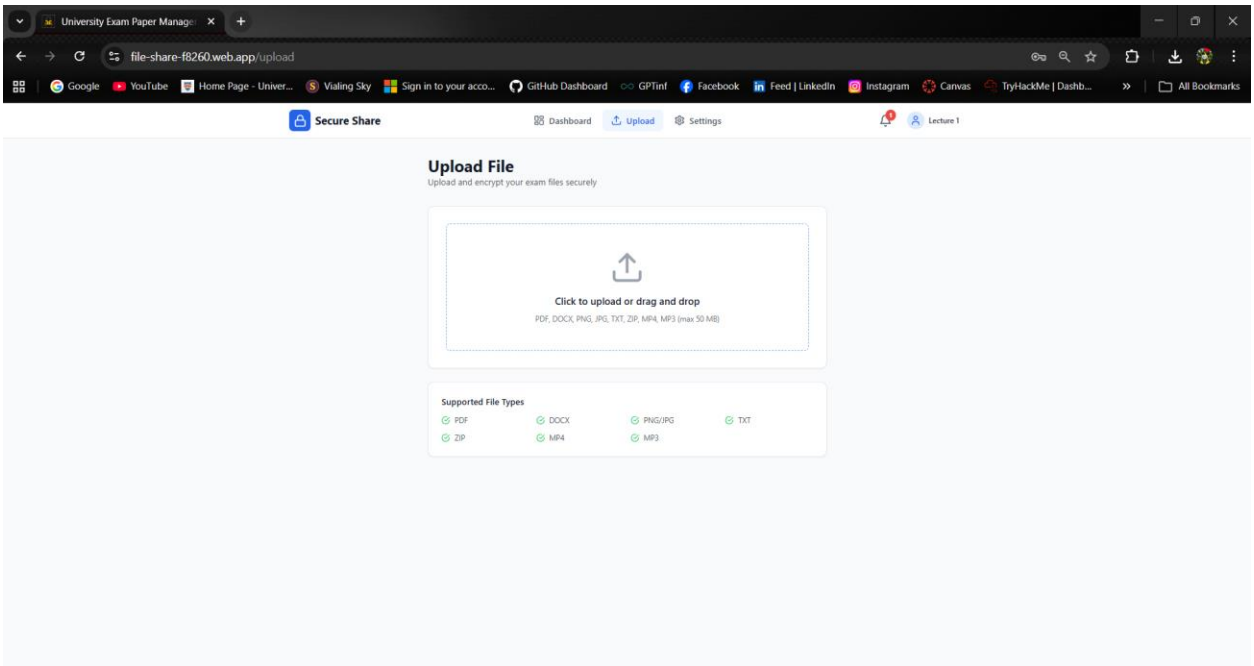

Figure 8 : File Upload Page

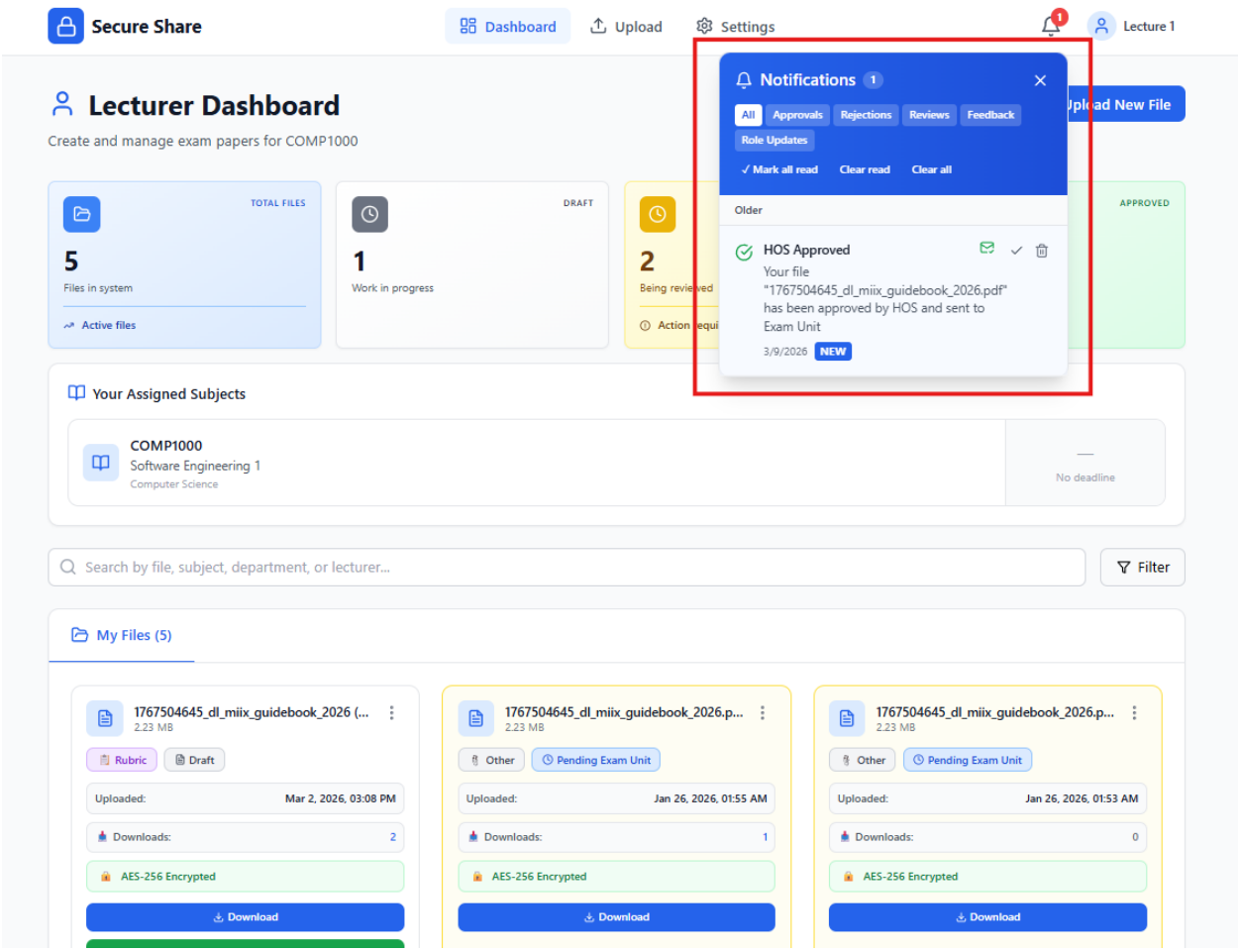

Figure 9 : Notification appear when action done / manipulated

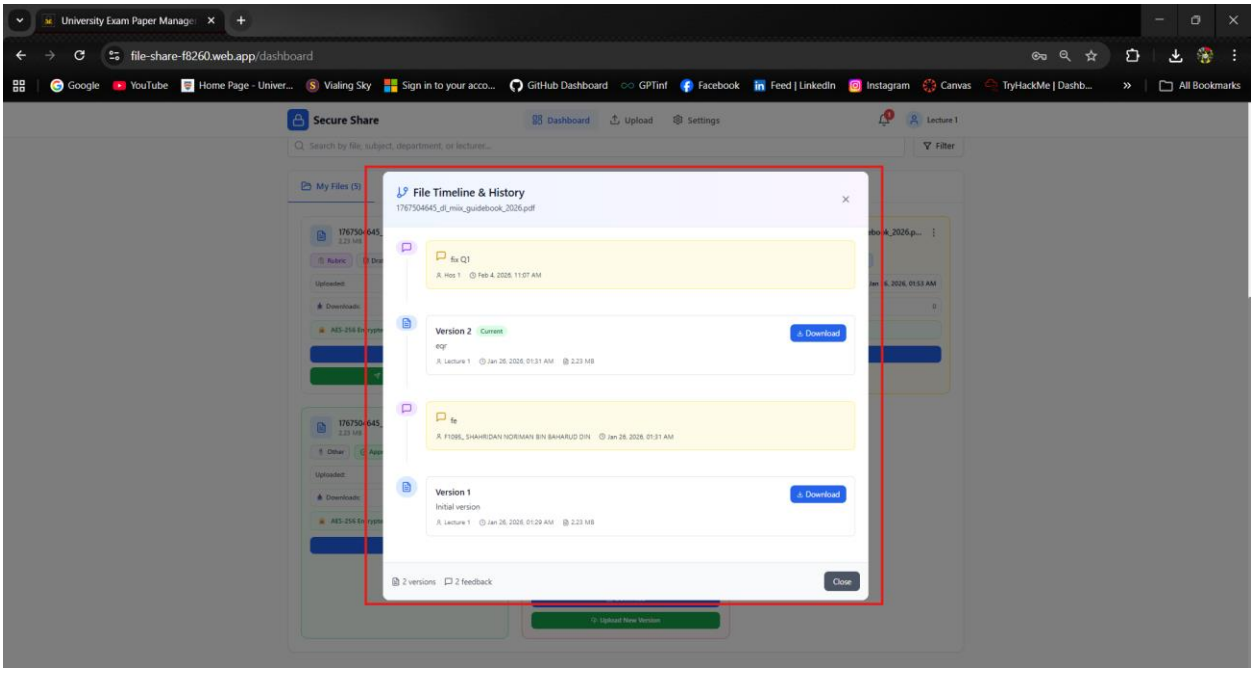

Figure 10 : File Timeline History Popup

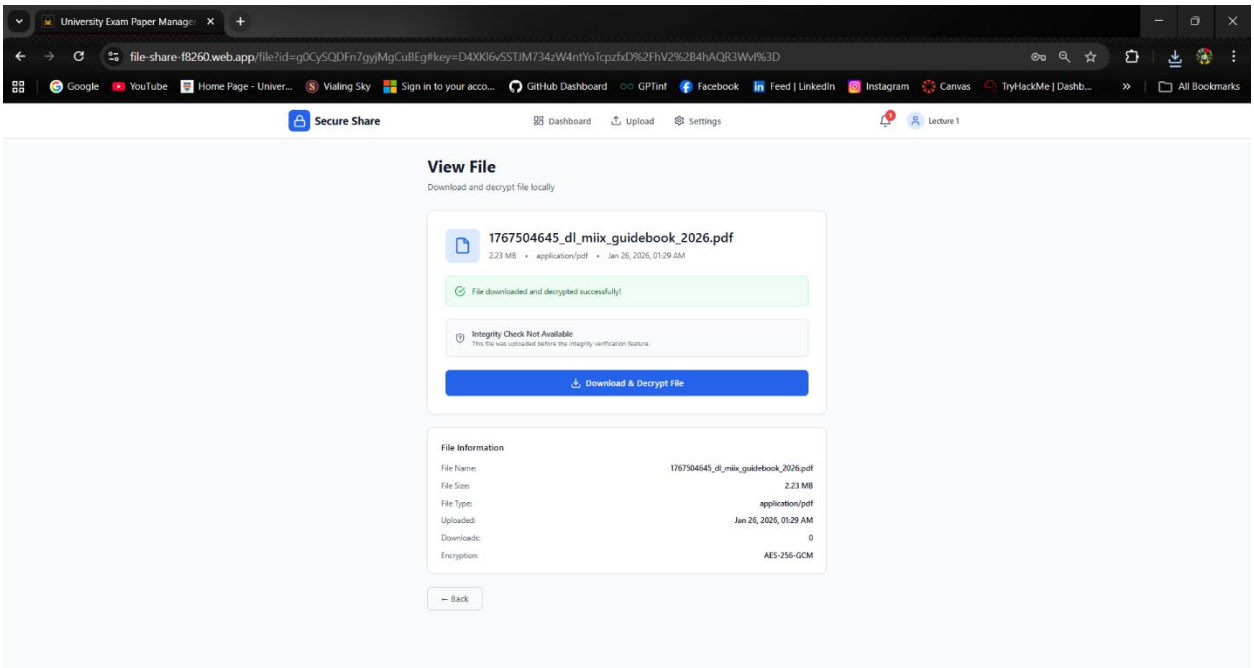

Figure 11 : View and Decrypt File page

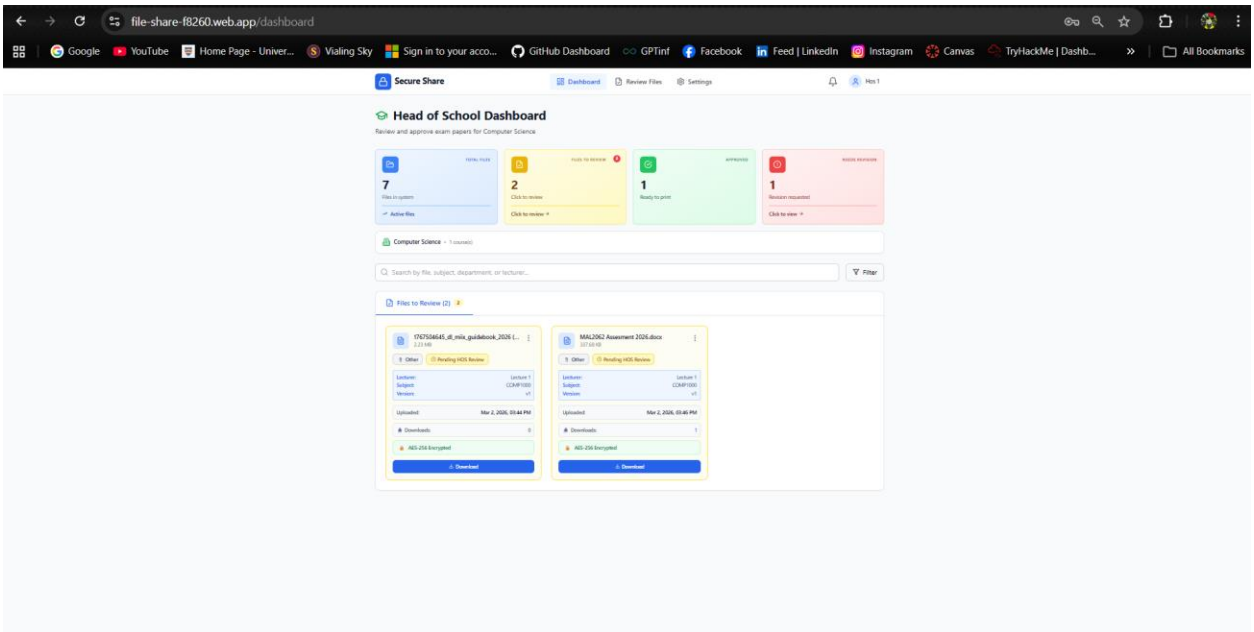

Figure 12 : Head of School (HOS) page

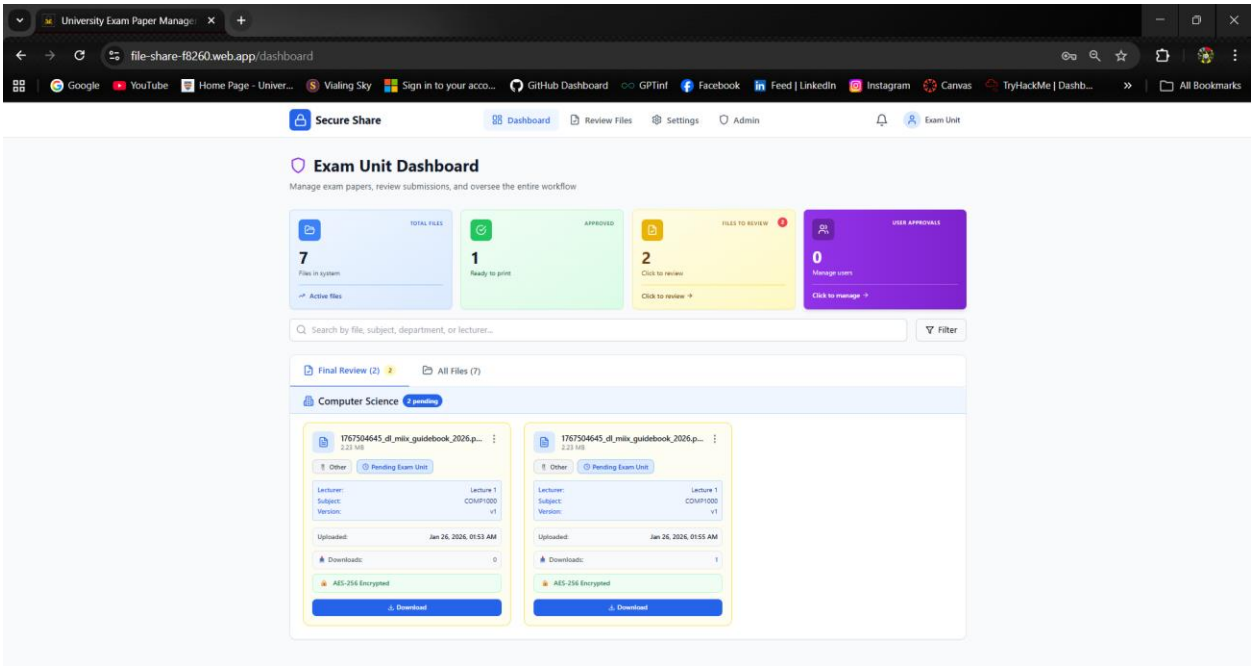

Figure 13 : Exam Unit (EU) page

The frontend handles all encryption and decryption operations directly in the browser using the Web Crypto API, ensuring that plaintext data never leaves the user's device.

7.1.2 Backend (Logic Tier)

Developed with **Firebase Cloud Functions** for serverless backend logic (e.g., email notification dispatch). The majority of business logic operations including CRUD operations, workflow status transitions, and notification creation are performed through direct Firestore SDK calls from the frontend, secured by Firestore Security Rules that enforce authentication and authorization server-side.

The service layer is organized into six dedicated modules:

Service Module	Responsibility
authService.js	User registration, login (email + Google OAuth), logout, password reset
encryptionService.js	AES-256-GCM encryption/decryption, PBKDF2 key derivation, SHA-256 hashing
storageService.js	Upload/download/delete encrypted files from Firebase Storage
firestoreService.js	All Firestore CRUD operations across 12 functional areas (2,100+ lines)
notificationService.js	Real-time notification management
emailService.js	Email notification delivery via Cloud Functions

7.1.3 Database (Data Tier)

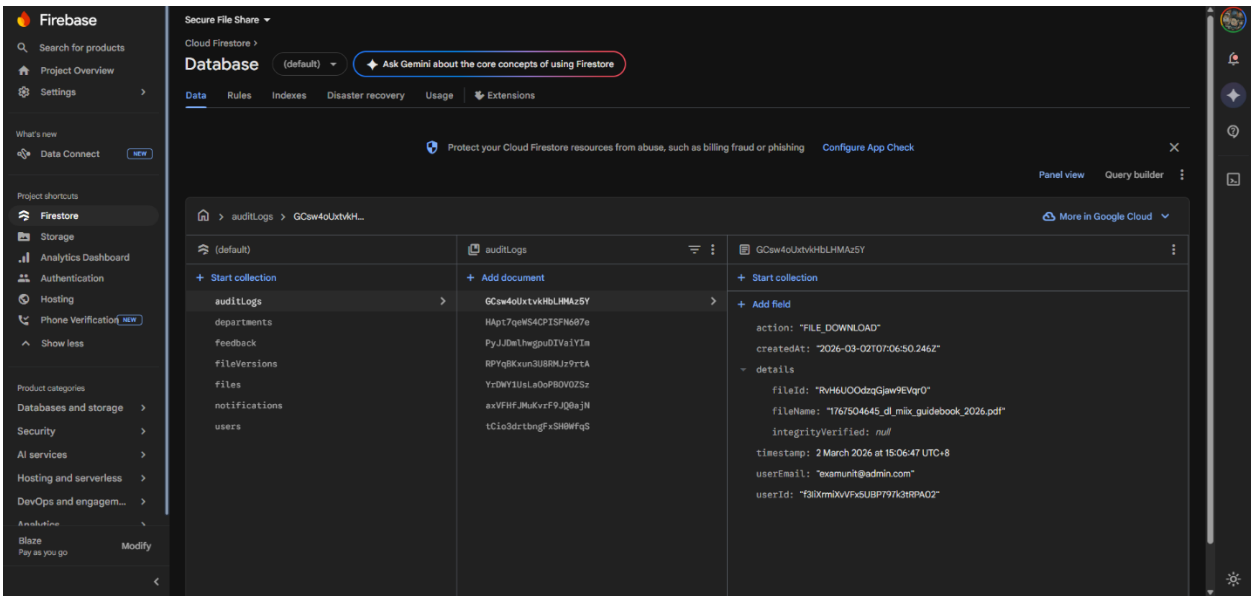

Figure 14 : Firebase Database page

Utilizing **Cloud Firestore** (NoSQL document database) for cloud-based storage of all application data. To maintain a true zero-knowledge architecture, Firebase strictly stores encrypted exam papers, metadata, and activity logs. **Firebase Storage** is used for storing the encrypted file blobs (binary data), while Firestore stores the structured metadata.

7.2 Security Design and Cryptography

Security is the cornerstone of the system's design. The platform implements a multi-layered security architecture:

7.2.1 End-to-End Encryption (E2EE)

The system implements **AES-256-GCM** (Advanced Encryption Standard with Galois/Counter Mode) encryption for all file uploads. The encryption parameters are defined as constants:

```
Encryption algorithm configuration
const ENCRYPTION_ALGORITHM = {
```

```
  name: 'AES-GCM',
  length: 256,        // 256-bit key length
  ivLength: 12        // 12-byte initialization vector
};
```

The encryption flow works as follows:

1. **Key Generation**: A unique 256-bit AES key is generated for each file using `crypto.subtle.generateKey()`.
2. **IV Generation**: A cryptographically random 12-byte Initialization Vector (IV) is generated using `crypto.getRandomValues()`.
3. **File Encryption**: The file's raw bytes are encrypted using `crypto.subtle.encrypt()` with AES-GCM mode.
4. **Data Packaging**: The IV is prepended to the encrypted data ([IV || ciphertext]) so it can be extracted during decryption.
5. **Upload**: Only the encrypted blob is uploaded to Firebase Storage with application/octet-stream content type and the .enc extension.

The decryption flow on download:

1. **Download**: The encrypted blob is fetched from Firebase Storage.
2. **IV Extraction**: The first 12 bytes are extracted as the IV, and the remaining bytes as the ciphertext.
3. **Key Retrieval**: The AES key is retrieved from the Firestore file metadata (protected by authentication rules).
4. **Decryption**: `crypto.subtle.decrypt()` decrypts the ciphertext using the key and IV.
5. **Integrity Verification**: The SHA-256 hash of the decrypted file is computed and compared to the hash stored at upload time.

7.2.2 Key Derivation (PBKDF2)

For password-based encryption of keys (used in file sharing), the system uses **PBKDF2** (Password-Based Key Derivation Function 2):

- **Algorithm**: PBKDF2 with SHA-256 hash function
- **Iterations**: 100,000 rounds (providing strong resistance against brute-force attacks)
- **Salt**: Per-file cryptographically random 16-byte salt (crypto.getRandomValues(new Uint8Array(16)))
- **Output**: 256-bit derived AES-GCM encryption key

The high iteration count of 100,000 ensures that each password guess requires significant computation, making brute-force attacks computationally infeasible.

7.2.3 File Integrity Verification (SHA-256)

To ensure that files have not been tampered with during storage or transmission, the system computes a **SHA-256 cryptographic hash** of the original file at upload time using `crypto.subtle.digest('SHA-256', data).` This hash is stored in the Firestore metadata alongside the file record. When a user downloads and decrypts a file, the system:

1. Recomputes the SHA-256 hash of the decrypted output
2. Compares it against the stored hash
3. Displays one of three statuses to the user:
    - **Integrity Verified** — Hash matches, file has not been tampered with
    - **Integrity Check Failed** — Hash mismatch, file may have been tampered with

- **Not Available** — File was uploaded before the integrity feature was implemented

7.2.4 Zero-Knowledge Architecture

The system operates on a zero-knowledge architecture where:

- Files are encrypted **before** upload using the browser's Web Crypto API
- The server only receives and stores **encrypted blobs** (.enc files)
- Encryption keys are stored in Firestore metadata, protected by authentication and role-based security rules, only authorized users (the file owner, the assigned HOS, and Exam Unit) can retrieve them through authenticated Firestore reads.
- Even if the Firebase Storage bucket were compromised, the attacker would only obtain encrypted ciphertext with no way to decrypt it without the corresponding AES-256 key from Firestore

7.3 Role-Based Access Control and Workflow Management

7.3.1 User Roles

To map to the university's operational hierarchy, the system incorporates strict **Role-Based Access Control (RBAC)** with department-level isolation. The system defines four user states:

Role	Permissions
pending	New registrations — can only view the Pending Approval page
lecturer	Upload files, view own files, submit for review, download approved files
hos (Head of School)	Review department files, approve/reject submissions, manage department lecturers
exam_unit	Full administrative access — manage users, departments, courses, subjects, deadlines, view audit logs

7.3.2 User Registration & Whitelist Auto-Approval

The system supports two registration methods:

1. **Email/Password Registration** — standard Firebase Auth registration with display name
2. **Google OAuth (Sign-in with Google)** — one-click sign-in using Google accounts

A **whitelist/pre-registration** system allows the Exam Unit to import staff from Excel spreadsheets via the BatchImportTab component. When a pre-registered email address signs up, the system:

- Automatically assigns the configured role (lecturer/HOS) and department
- Skips the pending approval queue
- Sends an auto-approval notification to the user
- Marks the whitelist entry as "claimed"

7.3.3 Three-Tier Approval Workflow

The system enforces a sequential approval workflow with the following status transitions:

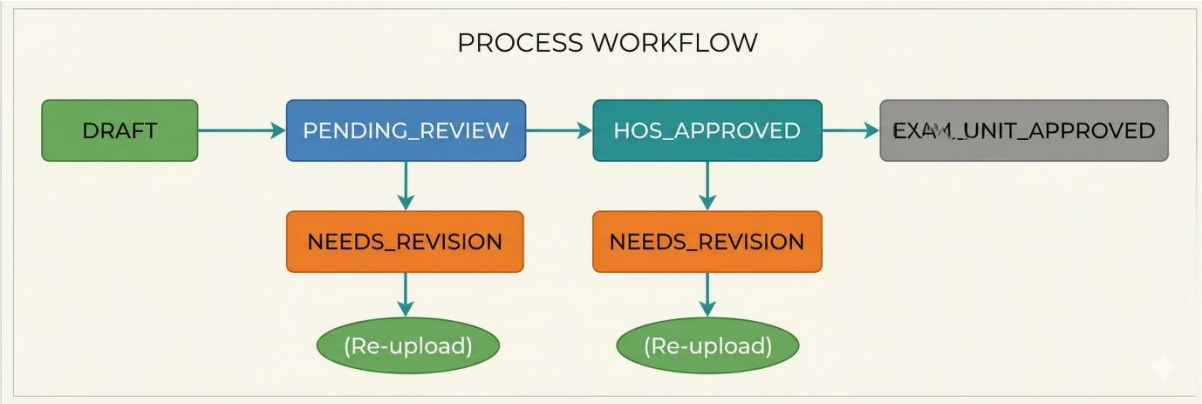

Figure 15 : Process Workflow

1. **Lecturer** uploads and encrypts the exam paper, which starts in DRAFT status. The lecturer submits it for review, transitioning to PENDING_REVIEW.
2. **Head of School (HOS)** reviews the document via the HOSReview page. The HOS can:
    - **Approve** → status becomes HOS_APPROVED, and the Exam Unit is notified
    - **Reject with feedback** → status becomes NEEDS_REVISION, and the lecturer is notified with the reason
3. **Exam Unit** receives the HOS-approved paper via the ExamUnitReview page. The Exam Unit can:
    - **Final Approve** → status becomes EXAM_UNIT_APPROVED, and the paper is ready for printing
    - **Reject with feedback** → status reverts to NEEDS_REVISION

At each transition, the system automatically dispatches notifications to the relevant users and logs the action in the audit trail.

7.3.4 File Versioning

The system supports **multi-version file management** via the fileVersions Firestore collection. When a lecturer revises a file after rejection:

- A new version record is created with its own encryption key and download URL
- The version number is incremented
- The previous versions remain accessible through the VersionHistoryModal component
- Each version tracks who uploaded it, when, and a description of changes

7.3.5 Deadline Management

The system implements a **cascading deadline system** with three granularity levels:

1. **Department-level deadline** — applies to all courses and subjects in the department
2. **Course-level deadline** — overrides the department deadline for that specific course
3. **Subject-level deadline** — overrides both course and department deadlines for that specific subject

The cascade resolution follows: Subject → Course → Department (most specific wins). When lecturers upload files past the deadline, the system automatically flags the submission as late (isLate: true).

7.4 Database Schema and Collections

The Firestore database is organized into the following collections:

7.4.1 Core Collections

Table 6 : Firebase stored metadata

Collection	Purpose	Key Fields
users	User profiles and role data	uid, email, displayName, role, department, createdAt, autoApproved
files	File metadata and workflow state	fileId, fileName, fileSize, fileType, encryptionKey, downloadURL, fileHash, workflowStatus, createdBy, departmentId, subjectId, version, isLate
fileVersions	Version history for each file	fileId, version, encryptionKey, downloadURL, uploadedBy, description
departments	Department, course, and subject hierarchy	name, code, courses[] (nested), deadlines
feedback	HOS/Exam Unit review comments	fileId, reviewerId, reviewerName, comments, action, attachmentURL
notifications	In-app notification delivery	userId, type, title, message, read, actionUrl, createdAt
auditLogs	Immutable accountability trail	userId, userEmail, action, details, timestamp, ipAddress
preRegisteredUsers	Whitelist for auto-approval	name, email, role, departmentId, claimed, claimedAt

7.4.2 Composite Indexes

The following Firestore composite indexes were created to optimize query performance:

Table 7 : Firestore composite indexes

Collection	Indexed Fields	Purpose
fileVersions	fileId ↑, version ↑	Efficient version history retrieval
files	departmentId ↑, workflowStatus ↑	HOS department file filtering
files	departmentId ↑, workflowStatus ↑, submittedForReviewAt ↓	HOS review queue sorted by submission date
files	workflowStatus ↑, submittedForReviewAt ↓	Exam Unit review queue
files	workflowStatus ↑, hosApprovedAt ↓	Exam Unit pending files sorted by HOS approval
notifications	userId ↑, createdAt ↓	User notification feed (newest first)
feedback	fileId ↑, createdAt ↓	File feedback history (newest first)

7.4.3 Audit Trails and Accountability

To guarantee accountability and compliance with data protection requirements, the system features comprehensive audit logging capabilities. The auditLogs collection records the following action types:

- FILE_UPLOAD — when a file is encrypted and uploaded
- FILE_DOWNLOAD — when a file is downloaded and decrypted (with integrity verification result)
- FILE_DELETE — when a file is permanently removed
- FILE_VIEW — when file metadata is accessed

Audit logs are immutable. The Firestore security rules enforce that no one (not even Exam Unit admins) can update or delete audit log entries:

```
match /auditLogs/{logId} {
  allow create: if isAuthenticated();
  allow read: if isExamUnit();
  allow update, delete: if false;   // Immutable —
cannot be modified or deleted
}
```

Only the Exam Unit role has read access to view all audit logs via the AdminPanel component's Audit Logs tab.

7.5 Firestore Security Rules

The system implements comprehensive server-side security through Firestore Security Rules, which enforce authorization at the database level:

7.5.1 Helper Functions

```
function isAuthenticated() { return request.auth !=
null; }
function getUserRole() { return
get(/databases/$(database)/documents/users/$(reques
t.auth.uid)).data.role; }
function isExamUnit() { return isAuthenticated() &&
getUserRole() == 'exam_unit'; }
function isHOS() { return isAuthenticated() &&
getUserRole() == 'hos'; }
function isLecturer() { return isAuthenticated() &&
getUserRole() == 'lecturer'; }
```

7.5.2 Key Security Rules

- **Role Self-Promotion Prevention**: Users can update their own profile but are explicitly blocked from modifying their role field

```
!request.resource.data.diff(resource.data).affectedKeys().hasAny(['role'])
```

- **New User Registration**: Only allows creating a user document with role: 'pending', preventing registration with elevated privileges
- **File Ownership**: Files can only be created by the authenticated user

```
(request.resource.data.createdBy == request.auth.uid)
```

- **Shared Access**: File read access checks if the requesting user's email is in the file's sharedWith array
- **FileVersion Access**: Cross-references the parent file's createdBy field to verify ownership
- **Department Isolation**: HOS users can only read and update files within their department

7.5.3 Firebase Storage Rules

Firebase Storage enforces additional security:

- Only authenticated users can **read** files (encrypted blobs)
- Only the file owner can **write** (upload) to their own user directory
- File size is capped at **50 MB** per upload
- Files are stored with .enc extension and application/octet-stream content type

7.6 Supported File Types

The system supports the following file formats for upload, each automatically encrypted before transmission:

Table 8 : Supported File Types

Format	MIME Type	Use Case
PDF	application/pdf	Exam papers, answer keys
DOCX	application/vnd.openxmlformats-officedocument.wordprocessingml.document	Draft papers, rubrics
DOC	application/msword	Legacy Word documents
PNG	image/png	Diagrams, scanned papers
JPG/JPEG	image/jpeg	Photos, scanned documents
TXT	text/plain	Plain text files
ZIP	application/zip	Bundled exam materials
MP4	video/mp4	Video-based assessments
MP3	audio/mpeg	Audio-based assessments (e.g., listening tests)

The maximum upload size is **50 MB**, enforced both on the client-side (via JavaScript validation) and server-side (via Firebase Storage security rules).

Chapter 8 : Quality Assurance

8.1 Overview

Quality assurance in a student project is an interesting challenge. On one hand, the testing infrastructure available to a small team with a semester-long timeline is naturally limited compared to what a professional development team might have. On the other hand, exam papers are genuinely sensitive documents, make cutting corners on quality especially uncomfortable. The team tried to strike a balance between comprehensive testing and practical constraints.

The QA strategy for Kunchee was organised around four main areas: functional correctness (does it do what it is supposed to do?), security (does the encryption actually work, and are there obvious vulnerabilities?), usability (can the intended users actually operate it?), and stability (does it break under normal usage conditions?). Not all of these could be tested with the same rigour, and the sections below are honest about where coverage was stronger and where it was thinner.

8.2 Testing Strategy

The overall testing approach followed the standard testing pyramid model: a base of unit tests covering individual functions, integration tests checking how modules work together, and a smaller number of system-level and user acceptance tests at the top. In practice, given the team's size and available time, the pyramid ended up somewhat flatter than ideal, there were fewer unit tests than would be ideal, and the integration and system testing took up a proportionally larger slice of the effort.

8.2.1 Unit Testing

Unit tests were written using Jest for the frontend React components and utility functions. The most important unit tests covered the encryption and decryption logic, since those

are the functions where a subtle bug would have the most serious consequences. A bug in the UI layout is annoying; a bug in the encryption is a security failure.

The encryption tests verified that:

- A file encrypted and then immediately decrypted with the correct key produces output byte-for-byte identical to the original
- Attempting to decrypt with a wrong key fails with an error rather than producing garbled output
- The encrypted output is different every time the same file is encrypted (due to the random IV used in AES-GCM)

The third test is worth explaining briefly. AES-GCM uses a randomly generated initialisation vector (IV) each time, which means encrypting the same plaintext twice produces two different ciphertexts. If the same key and IV combination were reused, an attacker could potentially recover information about the plaintext. The tests confirmed that the IV was being generated fresh for each encryption operation.

Beyond encryption, unit tests covered the form validation logic (email format, password strength requirements) and the role-checking functions used in the access control layer.

8.2.2 Integration Testing

Integration tests used Mocha to verify that the frontend and backend components worked correctly together. The most critical integration scenarios were:

1. A user logs in, uploads a file, and a correctly encrypted version appears in Firebase Storage
2. The workflow state changes correctly when a HOS approves or rejects a submission
3. An audit log entry is created for each of the above actions, with the correct user ID and timestamp
4. A user downloading a file receives the correct ciphertext, which the client successfully decrypts

Integration testing caught a couple of issues that unit tests would not have found in isolation. There was a race condition in the workflow state update under certain timing conditions, the Firestore document update and the audit log write could complete in the wrong order, resulting in a log entry with a timestamp slightly earlier than the state change it was supposed to record. This was addressed by restructuring the relevant code to use a Firestore transaction, which ensures atomicity.

8.2.3 System and End-to-End Testing

System-level testing was conducted manually, working through a set of scripted test scenarios that traced the full workflow from file creation to delivery at the Examination Unit. These scenarios included both happy-path cases (everything works as expected) and a selection of error cases (wrong password, upload of an oversized file, attempting to access a route without the required role).

The error cases were arguably more valuable than the happy-path tests, because they revealed how the system behaved under conditions the team had not fully anticipated during design. For instance, attempting to navigate directly to the HOS review page while logged in as a lecturer initially produced a blank screen rather than a meaningful 'access denied' message. This was caught during system testing and fixed before the final prototype was deployed.

8.2.4 Security Testing

Security testing used OWASP ZAP for automated scanning, supplemented by manual checks. The automated scan was run against the deployed prototype at file-share-f8260.web.app and identified a small number of medium-severity findings, mainly relating to HTTP response headers. These were addressed by configuring the appropriate security headers in the Firebase Hosting configuration such as Content-Security-Policy, X-Frame-Options, and X-Content-Type-Options.

Manual security checks focused on the areas where automated scanners are weakest: the application logic itself. The team specifically tested:

- Whether a lecturer could access another lecturer's files by manipulating the file ID in the URL (they could not, because Firestore security rules enforce ownership checks server-side)
- Whether a user could skip the 2FA step by replaying a valid session token (they could not, because 2FA completion is tracked in the session state)
- Whether a non-admin user could invoke admin-only API endpoints directly (they could not, because role verification happens on the server, not just the client)

It is worth being transparent about what the security testing did not cover. The team did not conduct a formal penetration test, and the codebase was not reviewed by anyone with professional security expertise. The testing described above provides reasonable confidence that the most obvious vulnerabilities are absent, but it would be an overstatement to describe the system as comprehensively hardened.

8.3 Usability Testing

Usability testing was conducted with a small group of three participants from outside the development team. Specifically, staff members at Peninsula College who were not involved in the project but whose roles are similar to the intended end users. Participants were asked to complete three tasks without any guidance from the team: log in using 2FA, upload a file, and check the audit log for their account.

The sessions were observed but not recorded. Notes were taken on where participants hesitated, made errors, or expressed confusion. Overall, the results were reasonably encouraging. All three participants completed all three tasks. That said, two of the three paused for a noticeable amount of time on the 2FA screen, unsure whether they needed to check their email or their phone. This pointed to a labelling issue in the interface rather than a fundamental workflow problem, and the text was updated accordingly.

One participant commented that the file upload confirmation was too quiet. A small success toast notification that appeared briefly in the corner was easy to miss. This kind of feedback is difficult to predict during development and only really surfaces when someone who was not involved in building the system tries to use it. The confirmation display was made more prominent in response.

8.4 Bug Tracking and Resolution

Bugs were tracked using GitHub Issues throughout the development process. Each issue was tagged with a severity label (critical, major, minor, cosmetic) and assigned to the relevant team member. The team agreed at the start of the project that no critical or major bugs would be left open at the point of final submission.

In total, 14 issues were raised during the testing phases. Of these, 11 were resolved before the final prototype build. The remaining three were classified as minor. Specifically, a visual layout issue on very small screens, a slightly inconsistent timestamp format in the audit log display, and a missing loading indicator on the file download button. None of these affected functionality or security, and they were documented in the known issues section of the technical documentation.

8.5 Limitations and Reflections on QA

Honest reflection on the QA process suggests a few areas that, given more time and resources, would benefit from additional attention.

First, test coverage was not measured systematically. The team could not say with confidence what percentage of the codebase was exercised by the unit and integration tests. In a professional project, a coverage target (often 80% line coverage or higher) would typically be set and tracked. This was not done here, partly due to the learning curve involved in configuring coverage tooling for the specific stack used.

Second, the usability testing sample was very small. Three participants is enough to surface the most obvious usability problems, but subtle issues may well remain. A more

thorough usability evaluation would involve a larger and more diverse group of participants, ideally including actual lecturers from the institution.

Third, the security testing was largely limited to the application layer. A more comprehensive assessment might examine the Firebase configuration in greater depth, review the key management approach against established cryptographic best practices, and assess the system's resilience against threats like credential stuffing or account takeover.

These limitations do not invalidate the testing that was conducted, but they are worth acknowledging. The system as tested provides meaningful security improvements over the USB-based approach it is intended to replace, and the core functionality has been verified to work correctly. The gaps identified above represent areas for future work rather than present failures.

Chapter 9 : Conclusion

9.1 Summary

This project successfully developed and implemented a **Secure University Exam Paper Management System** that addresses critical challenges in academic document security and workflow management. The system demonstrates the practical integration of advanced cryptographic techniques with user-friendly interfaces, providing universities with a production-ready solution for managing sensitive examination materials. Through the implementation of end-to-end encryption using AES-256-GCM, a three-tier approval workflow, and comprehensive audit capabilities, the project achieved its primary objective of balancing robust security with operational efficiency.

The developed system incorporates all planned core features, including client-side encryption that ensures files never exist in plaintext on servers, role-based access control with department-level isolation, complete version control with download history tracking, and real-time notifications for workflow management. The successful deployment at https://file-share-f8260.web.app with a functional demo environment validates the system's readiness for real-world academic deployment. Test accounts and comprehensive documentation further demonstrate the project's maturity and attention to practical implementation details.

9.2 Technical and Practical Outcomes

From a technical perspective, the project successfully demonstrated the feasibility of implementing enterprise-grade security using modern web technologies. The Web Crypto API integration enabled true zero-knowledge architecture where encryption keys remain under user control, eliminating server-side data breach risks. The Firebase infrastructure provided scalable backend services while maintaining security through custom Firestore rules and storage policies. The React-based frontend achieved the critical balance between security transparency and user accessibility, proving that sophisticated

cryptography can be integrated into intuitive interfaces suitable for non-technical academic staff.

Practically, the system addresses real operational challenges faced by universities. The three-tier approval workflow (Lecturer → Head of School → Exam Unit) directly aligns with typical academic governance structures, while the department isolation ensures data segregation and compliance with privacy requirements. The comprehensive audit trails and version control capabilities provide the accountability measures essential for regulatory compliance and institutional governance.

9.3 Final Reflections

This Computing Group Project (MAL2020) successfully demonstrated that sophisticated security solutions can be both technically robust and practically implementable. The Secure University Exam Paper Management System represents more than just a software application. It embodies the principle that strong cryptography and user-centric design are not mutually exclusive but can be synergistically combined to create solutions that protect sensitive academic assets while supporting efficient institutional workflows.

The project's success validates the importance of interdisciplinary approaches that combine computer science expertise with domain knowledge of academic processes. By addressing real-world challenges in exam paper management, the system contributes to the broader goal of enhancing academic integrity and institutional security. The comprehensive documentation, testing, and deployment achievements demonstrate the team's ability to deliver production-quality software that meets enterprise requirements.

Ultimately, this project serves as a proof-of-concept that universities can implement enterprise-grade security for sensitive documents without compromising usability or operational efficiency. The successful integration of modern web technologies with cryptographic best practices provides a foundation for future developments in secure academic document management, potentially influencing how institutions worldwide approach the protection of sensitive educational materials.

References

Beck, K. et al. (2001) Manifesto for Agile Software Development. Available at: https://agilemanifesto.org (Accessed: 25 March 2026).

Chacon, S. and Straub, B. (2014) Pro Git. 2nd edn. New York: Apress. Available at: https://git-scm.com/book/en/v2 (Accessed: 25 March 2026).

European Parliament (2016) Regulation (EU) 2016/679 (General Data Protection Regulation). Official Journal of the European Union, L 119, pp. 1–88.

Malaysia (2010) Personal Data Protection Act 2010 (Act 709). Kuala Lumpur: Attorney General's Chambers of Malaysia.

NIST (2001) FIPS 197: Advanced Encryption Standard (AES). Gaithersburg, MD: National Institute of Standards and Technology.

NIST (2007) SP 800-38D: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. Gaithersburg, MD: National Institute of Standards and Technology.

NIST (2023) SP 800-132: Recommendation for Password-Based Key Derivation. Gaithersburg, MD: National Institute of Standards and Technology.

OWASP Foundation (2021) OWASP Top Ten. Available at: https://owasp.org/www-project-top-ten/ (Accessed: 25 March 2026).

Sandhu, R.S. et al. (1996) 'Role-based access control models', IEEE Computer, 29(2), pp. 38–47.

Sommerville, I. (2016) Software Engineering. 10th edn. Harlow: Pearson Education.

W3C (2023) Web Cryptography API. Available at: https://www.w3.org/TR/WebCryptoAPI/ (Accessed: 25 March 2026).

Appendices

Appendix A: Online Zoom Meetings with Group Member

Appendix B: Meeting Picture with Group Member

Appendix C: Regular project meetings on sprints

Appendix D: Minutes of meetings done when meetings with stakeholders

Kunchee

Location: Barcelona – Francis Light Library
Date: 24/10/2025
Time: 2:30 p.m.

Meeting Minutes
Agenda items

I. Project overview and objectives
II. Client requirements clarification
III. System design and user interface discussion
IV. Database and backend planning
V. Discussing workflow in exam unit

Action Items	Owner(s)	Deadline	Status
Privacy control	Farah	[28/10/2025]	In Progress
User Interface	Danial & Wayden	[TBC]	In Progress
Function	Shahridan	[TBC]	In Progress

VI. **Privacy and Security Disclaimer**

Kunchee is designed as a **secure note and code snippet sharing web application** developed for the **Exam Unit of The Ship Campus** to ensure confidentiality, integrity, and controlled access to examination-related materials and communications.

By using Kunchee, all stakeholders agree to the following privacy and security terms:

1. **Data Confidentiality**
   All information, notes, and code snippets shared through Kunchee are strictly confidential and accessible only to authorized users designated by the Exam Unit. Unauthorized access, distribution, or disclosure of any stored content is strictly prohibited.
2. **End-to-End Encryption**
   Kunchee implements end-to-end encryption to protect data in transit and at rest. The system ensures that only the sender and intended recipient can decrypt the shared information.

Kunchee

Location: Barcelona – Francis Light Library
Date: 24/10/2025
Time: 2:30 p.m.

3. **User Authentication and Access Control**
   Each user is required to log in using a verified institutional account. Role-based access controls are enforced to limit permissions according to user responsibilities within the Exam Unit.
4. **Data Integrity and Storage**
   All stored data is protected from unauthorized modification or tampering. Regular integrity checks and audit logs are maintained to track system access and changes.
5. **Liability and Responsibility**
   While Kunchee strives to maintain the highest level of data security, the development team and The Ship Campus are not liable for damages caused by user negligence, such as sharing credentials or misuse of the system.
6. **Compliance with Institutional Policy**
   The application adheres to The Ship Campus' data protection and cybersecurity policies, ensuring alignment with ethical, academic, and regulatory standards.

Created by:		**Authorized by:**	
Signature:		Signature:	
Name:	Nur Farah Aqilah	Name:	Farahratul Shida
Date:	24/10/2025	Date:	24/10/2025

Figure 1: Minutes of Meeting (24/10/2025)

Appendix E: Minutes of meetings done when meetings with stakeholders

Kunchee

Location: Barcelona – Francis Light Library
Date: 24/10/2025
Time: 2:30 p.m.

Meeting Minutes
Agenda items

I. Project overview and objectives
II. Client requirements clarification
III. System design and user interface discussion
IV. Database and backend planning
V. Discussing workflow in exam unit

Action Items	Owner(s)	Deadline	Status
Privacy control	Farah	[28/10/2025]	In Progress
User Interface	Danial & Wayden	[TBC]	In Progress
Function	Shahridan	[TBC]	In Progress

VI. **Privacy and Security Disclaimer**

Kunchee is designed as a **secure note and code snippet sharing web application** deve'oped for the **Exam Unit of The Ship Campus** to ensure confidentiality, integrity, and controlled access to examination-related materials and communications.

By using Kunchee, all stakeholders agree to the following privacy and security terms:

1. **Data Confidentiality**
   All information, notes, and code snippets shared through Kunchee are strictly confidential and accessible only to authorized users designated by the Exam Unit. Unauthorized access, distribution, or disclosure of any stored content is strictly prohibited.
2. **End-to-End Encryption**
   Kunchee implements end-to-end encryption to protect data in transit and at rest. The system ensures that only the sender and intended recipient can decrypt the shared information.

Kunchee

Location: Counseling Room – Francis Light Library
Date: 19/11/2025
Time: 2:30 p.m.

2. **End-to-End Encryption**
   Kunchee implements end-to-end encryption to protect data in transit and at rest. The system ensures that only the sender and intended recipient can decrypt the shared information.
3. **User Authentication and Access Control**
   Each user is required to log in using a verified institutional account. Role-based access controls are enforced to limit permissions according to user responsibilities within the Exam Unit.
4. **Data Integrity and Storage**
   All stored data is protected from unauthorized modification or tampering. Regular integrity checks and audit logs are maintained to track system access and changes.
5. **Liability and Responsibility**
   While Kunchee strives to maintain the highest level of data security, the development team and The Ship Campus are not liable for damages caused by user negligence, such as sharing credentials or misuse of the system.
6. **Compliance with Institutional Policy**
   The application adheres to The Ship Campus' data protection and cybersecurity policies, ensuring alignment with ethical, academic, and regulatory standards.

Created by:		**Authorized by:**	
Signature:		Signature:	
Name:	Nur Farah Aqilah	Name:	Farahzatul Shida
Date:	19/11/2025	Date:	19/11/2025

Figure 2: Minutes of Meeting (19/11/2025)

Appendix F: Minutes of meetings done when meetings with stakeholders

Kunchee

Location: Counseling Room – Francis Light Library
Date: 11/02/2026
Time: 2:00p.m.

Meeting Minutes
Agenda items

I. System Demonstration & Client Experience

- **Full System Walkthrough:** Displaying the updated flow from login to secure note sharing.
- **Verification of Previous Requests:** Showing how previous feedback has been implemented.
- **Privacy Controls:** Demonstrated the finalized privacy settings to ensure exam data is hidden from unauthorized users.

II. Discussion of Client Feedback

- **Client Satisfaction:** The client acknowledged the improvements made since the last session and verified that the core functions meet the Exam Unit's needs.
- **Final Requests for Improvement:** The client requested the following final adjustments before the official handover:

  1. **Feature Addition (Deadline for Submission):** The system must include a "Deadline" setting.

III. Privacy and Security Disclaimer

Kunchee is designed as a **secure file and code snippet sharing web application** developed for the **Exam Unit of The Ship Campus** to ensure confidentiality, integrity, and controlled access to examination-related materials and communications.

By using Kunchee, all stakeholders agree to the following privacy and security terms:

1. **Data Confidentiality**
   All information, notes, and code snippets shared through Kunchee are strictly confidential and accessible only to authorized users designated by the Exam Unit. Unauthorized access, distribution, or disclosure of any stored content is strictly prohibited.
2. **End-to-End Encryption**
   Kunchee implements end-to-end encryption to protect data in transit and at rest. The system ensures that only the sender and intended recipient can decrypt the shared information.

Kunchee

Location: Counseling Room – Francis Light Library
Date: 11/02/2026
Time: 2:00p.m.

3. **User Authentication and Access Control**
   Each user is required to log in using a verified institutional account. Role-based access controls are enforced to limit permissions according to user responsibilities within the Exam Unit.
4. **Data Integrity and Storage**
   All stored data is protected from unauthorized modification or tampering. Regular integrity checks and audit logs are maintained to track system access and changes.
5. **Liability and Responsibility**
   While Kunchee strives to maintain the highest level of data security, the development team and The Ship Campus are not liable for damages caused by user negligence, such as sharing credentials or misuse of the system.
6. **Compliance with Institutional Policy**
   The application adheres to The Ship Campus' data protection and cybersecurity policies, ensuring alignment with ethical, academic, and regulatory standards.

Created by:		**Authorized by:**	
Signature:		Signature:	
Name:	Nur Farah Aqilah	Name:	Farahzatul Shida
Date:	11/02/2026	Date:	11/02/2026

Figure 3: Minutes of Meeting (11/02/2026)

Appendix G: Minutes of meetings done when meetings with stakeholders

Kunchee

Location: Counseling Room – Francis Light Library
Date: 12/03/2026
Time: 9:00a.m.

Meeting Minutes
Agenda items

I. **System Demonstration & Client Experience**
- **New Function Walkthrough**: Demonstration of a new function that has already been implemented in the system as requested by client in the past meeting.
- Project handover comments and signage.

II. **Discussion of Client Feedback**
- **Client Satisfaction**: The client acknowledged the improvements made since the last session and verified that the new functions meet the Exam Unit's needs.
- **Requirements Verification:** The client officially verified that the system's functionalities fully meet the operational requirements and expectations of The Ship Campus Exam Unit.

III. **Privacy and Security Disclaimer**

Kunchee is designed as a **secure file and code snippet sharing web application** developed for the **Exam Unit of The Ship Campus** to ensure confidentiality, integrity, and controlled access to examination-related materials and communications.

By using Kunchee, all stakeholders agree to the following privacy and security terms:

1. **Data Confidentiality**
   All information, notes, and code snippets shared through Kunchee are strictly confidential and accessible only to authorized users designated by the Exam Unit. Unauthorized access, distribution, or disclosure of any stored content is strictly prohibited.
2. **End-to-End Encryption**
   Kunchee implements end-to-end encryption to protect data in transit and at rest. The system ensures that only the sender and intended recipient can decrypt the shared information.
3. **User Authentication and Access Control**
   Each user is required to log in using a verified institutional account. Role-based access controls are enforced to limit permissions according to user responsibilities within the Exam Unit.

Kunchee

Location: Counseling Room – Francis Light Library
Date: 12/03/2026
Time: 9:00a.m.

4. **Data Integrity and Storage**
   All stored data is protected from unauthorized modification or tampering. Regular integrity checks and audit logs are maintained to track system access and changes.
5. **Liability and Responsibility**
   While Kunchee strives to maintain the highest level of data security, the development team and The Ship Campus are not liable for damages caused by user negligence, such as sharing credentials or misuse of the system.
6. **Compliance with Institutional Policy**
   The application adheres to The Ship Campus' data protection and cybersecurity policies, ensuring alignment with ethical, academic, and regulatory standards.

Created by:		**Authorized by:**	
Signature:		Signature:	
Name:	Nur Farah Aqilah	Name:	Farahanul Shida
Date:	12/03/2026	Date:	12/03/2026

Figure 4: Minutes of Meeting (12/03/2026)

Appendix H: Meetings with our stakeholders (The Ship Campus Exam Unit representatives)